Operating System Concepts

Introduction

OS Contents

- OS Concepts
- Linux commands
- Shell Scripts
- Linux System Call Programming

Schedules

- Lecture: 8.00 am to 1.30 pm (5 days)
    - Break1: 9:45 am (25 mins)
    - Break2: 12:00 pm (10 mins)
- Lecture: 2.30 pm to 4.15 pm (3 days)
- Lab: 4.40 pm to 7.00 pm (3 days)

Trainer

- Nilesh Ghule
- M.Sc. Electronics Science
- Experience: 20+ Years
- Subjects: Operating Systems, Device Drivers, Microcontrollers, Java, Advanced Java, Databases, Big Data.

Agenda

- Introduction
- What is OS?
- OS Functions
- Process management

- File management
- User interfacing

Learning OS

- step 1: End user
  - Linux commands
- step 2: Administrator
  - Install OS (Linux)
  - Configuration - Users, Networking, Storage, ...
  - Shell scripts
- step 3: Programmer
  - Linux System call programming
- step 4: Designer/Internals
  - OS/UNIX & Linux internals

What is OS?

- Interface between end user and computer hardware.
- Interface between Programs and computer hardware.
- Control program that controls execution of all other programs.
- Resource manager/allocator that manage all hardware resources.
- Bootable CD/DVD = Core OS + Applications + Utilities
- Core OS = Kernel -- Performs all basic functions of OS.

OS Functions

- CPU scheduling
- Process Management
- Memory Management
- File & IO Management
- Hardware abstraction
- User interfacing

- Security & Protection
- Networking

Hardware abstraction

- OS hides all hardaware intricacies from the end user as well as user programs.
- Same program can work for different hardware components. e.g. Same Hello World program for CRT monitors, LCD monitors, or LED monitors.

Process Management

Program

- Set of instructions given to the computer --> Executable file.
- Program --> Sectioned binary --> "objdump" & "readelf".
    - Exe header --> Magic number, Address of entry-point function, Information about all sections. (objdump -h program.out)
    - Text --> Machine level code (objdump -S program.out)
    - Data --> Global and Static variables (Initialized)
    - BSS --> Global and Static variables (Uninitialized)
    - RoData --> String constants
    - Symbol Table --> Information about the symbols (Name, Size, section, Flags, Address) (objdump -t program.out)
- Program (Executable File) Format
    - Windows -- PE (Portable Executable)
    - Linux -- ELF (Executable Linking Format)
- Programs are stored on disk (storage).

Process

- Progam under execution.
- Process execute in RAM.
- Process has multiple sections i.e. text, data, rodata, heap, stack. ... into user space and its metadata is stored into kernel space in form of PCB struct.
- Process control block contains information about the process (required for the execution of process).
    - Process id
    - Exit status

        - 0 - Indicate successful execution
        - Non-zero - Indicate failure
    - Scheduling information (State, Priority, Sched algorithm, Time, ...)
    - Memory information (Base & Limit, Segment table, or Page table)
    - File information (Open files, Current directory, ...)
    - IPC information (Signals, ...)
    - Execution context (Values of CPU registers)
    - Kernel stack
- PCB is also called as process descriptor (PD), uarea (UNIX), or task_struct (Linux).
- In Linux, size of task_struct is more than 5 kb.

File Management

File

- File is collection of data/information on storage device.
    - File = Contents (Data) + Information (Metadata)
    - The data is stored in zero or more Data blocks (in FS), while metadata is stored in the FCB (in filesystem).
- FCB is called as "inode" on UNIX/Linux. It contains
    - type: UNIX/Linux has 7 types of files
        - -: regular, d: directory, l: symbolic link, p: pipe, s: socket, c: char device, b: block device
    - size: number of bytes
    - links: number of hard links
    - mode (permissions): (u) rwx, (g) rwx, (o) rwx
    - user & group
    - time-stamps: modification, creation, access.
    - info about data blocks
- terminal> ls -l
    - type, mode, links, user, group, size, timestamp, name.
    - File Types
        - Regular file (-)
        - Directory file (d)

        - Link file (l)
        - Socket file (s)
        - Pipe file (p)
        - Character Special file (c)
        - Block Special file (b)
- terminal> stat filepath

File System

- Files are stored on storage device. Arrangement of files in storage device is called as "File System".
- e.g. FAT, NTFS, EXT2/3/4, ReiserFS, XFS, HFS, etc.
- File System logically divide partition into 4 sections.
    - Boot block/Boot sector
        - Contains programs/info required for booting of OS
        - Typically contains bootstrap program and bootloader program
    - Super block/Volume control block
        - Contains information of whole partition.
        - Capacity, Label.
        - terminal> df -h
            - Total number of data blocks/inodes.
            - Number of used/free data blocks/inodes.
            - Information of free data blocks/inodes.
    - Inode List/Master file table
        - Inodes (FCB) for each file
    - Data blocks
        - Stores data of the file.
        - Each file have zero or more data blocks.
        - Size of data blocks can be configured while creating file system
- File system is created by the format utility while formatting the partition.

    - Windows: format.exe
    - Linux: mkfs
        - terminal> sudo mkfs -t ext3 /dev/sdb1
        - terminal> sudo mkfs -t vfat /dev/sdb1
        - -t fs_type e.g. ext3, ext4, vfat, ntfs, ...
        - partition e.g. /dev/sdb1

- Disk/partition naming conventions

    - Windows:
        - Disks are named as disk0, disk1, ...
        - partitions are named as drives i.e. C:, D:, E:, ...
    - Linux:
        - Disks are named as /dev/sda, /dev/sdb, /dev/sdc, etc.
        - Partitions per disk are named as
            - sda partitions: sda1, sda2, sda3, ...
            - sdb partitions: sdb1, ...

Linux File Structure

- Linux follows “/” (root) file system.
- “/” is a starting point of Linux file system.
- All your data is stored in this partition.
- / contains boot, bin, sbin, etc, root, home, dev, proc, mnt, media, opt
- In Linux everything is a file.
- Mainly there are two types of files in Linux
    - File
    - Directory (Folder)
- Linux Directories
    - boot - files related to booting
        - vmlinuz - kernel Image
        - grub - boot loader
        - config - kernel configuration

        - initrd/initramfs - initail root file system
    - bin - user commands in binary format
    - sbin - all admin/system commands in binary format
    - lib - shared program libraries required by kernel
    - etc - configuration files
    - root - home directory of root user
    - home - it contains sub directories for each user with its name
        - nilesh -> /home/nilesh
        - sunbeam -> /home/sunbeam
        - osboxes -> /home/osboxes
    - usr - read only directory that stores small programs and files accessible to all users
    - dev - it contains all device related files
    - proc - virtual file system - it contains information about system or processes
    - sys - entries of each block devices, subdirectories for each physical bus type supported, every device class registered with the kernel, global device hierarchy of all devices
    - mnt - it is temporary mount point
    - media - it is mount point for media eg cdrom
    - opt - stores optional files of large softwares
    - tmp - temporary files that may be lost on system shutdown

User interfacing

- UI of OS is a program (Shell) that interface between End user and Kernel.
- Shell -- Commmand interpreter
    - End user --> Command --> Shell --> Kernel
- User interfacing (Shell)
    - Graphical User Interface (GUI)
    - Command Line Interface (CLI)

Example shells

- Windows

    - GUI shell: explorer.exe
    - CLI shell: cmd.exe, powershell.exe
- DOS
    - CLI shell: command.com
- Unix/Linux
    - CLI shell: bsh, "bash", ksh, csh, zsh, ...
        - ls /bin/*sh
        - echo $SHELL
    - shell of current user can be changed using "chsh" command.
- GUI shell/standards
    - GNOME: GNU Network Object Model Environment (e.g. Ubuntu, Redhat, CentOS, ...)
    - KDE: Kommon Desktop Environment (e.g. Kubuntu, SuSE, ...)

Path

- It is a unique location of any file in the file system.
- It is represented by character strings with few delimiters (“/”, “\”, “:”)
- Types of path
    - There are two types of paths in linux
    - Absolute path
        - Path which starts with “/” is called as absolute path.
        - E.g. /home/nilesh/MyData/Demos/demo01.sh
    - Relative path
        - Path with respect to current directory is called as relative path
        - E.g. MyData/Assignments/assign02.pdf

Linux Commands

- env -- display all env variables

- echo $VAR -- display given env variable
- which programname -- find the program in all directories mentioned in PATH variable and display its path if found.
- pwd -- prints current/present working directory
- man command -- show help of the command
    - press "q" to exit.
- ls -- shows contents of current of current directory
    - -l: long listing (details of file/subdirectories)
    - -i: show inode number
    - -R: recursive listing
    - -S: sorted order of size (desc sort)
    - -r: reverse order used with sorting
    - -a: hidden + non-hidden + . + ..
    - -A: hidden + non-hidden
- ls dirpath -- shows contents of given directory
    - ls /
    - ls /home
    - ls /dev
- mkdir dirpath -- create new directory
    - mkdir /tmp/movies
    - mkdir /tmp/songs
    - cd /tmp
    - mkdir movies/hw
    - cd movies
    - mkdir bw
    - mkdir -p /tmp/songs/classic/gazal
    - ls -R /tmp/movies
    - ls -R /tmp/songs
- cat > filepath
    - enter file contents and press ctrl+d to exit.
    - create new file if doesn't exist
    - truncate if file exist

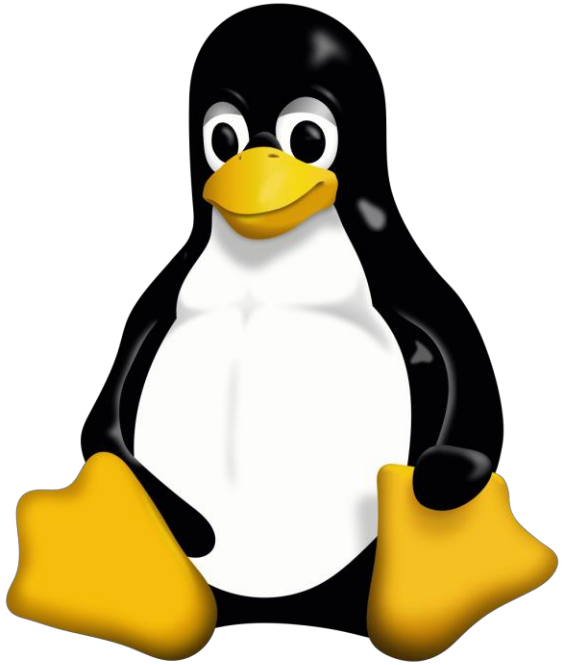
- cat filepath
    - display file contents
    - -n: show line numbers
- cat >> filepath
    - append to file
- tac filepath
- rev filepath
- cp srcfilepath destdirpath
    - cp hw/inception.txt bw
- cp srcfilepath destfilepath
    - cd hw
    - cp inception.txt inception2.txt
- cp -R srcdirpath destdirpath
    - cp -R /tmp/movies /home/nilesh/mar-24/dac/os/day01
- mv filepath destdirpath
    - mv /home/nilesh/mar-24/dac/os/day01/a.out /tmp
    - ls /tmp/*.out
    - /tmp/a.out
- mv dirpath destdirpath
    - ls /tmp/songs
    - mv /tmp/songs/ /home/nilesh/mar-24/dac/os/day01
    - ls /tmp/songs
- mv filename newfilename
    - cd /home/nilesh/mar-24/dac/os/day01/movies/bw
    - ls
    - mv inception.txt incept.txt
        - file rename
    - cat incept.txt
    - mv incept.txt .incept.txt
    - ls -A
    - cat .incept.txt

    - mv .incept.txt incept.txt
    - ls
- rmdir dirpath
    - can delete only empty directories
    - rmdir /home/nilesh/mar-24/dac/os/day01/movies
        - Error
- rm filepath
    - delete file
    - cd /home/nilesh/mar-24/dac/os/day01
    - rm movies/bw/incept.txt
- rm -R dirpath
    - delete directories with all its contents
    - rm -R movies
- touch filepath
    - If file exists, change its timestamp.
    - If file doesn't exist, create empty file
- head -n filepath
    - Display first n lines of file
- tail -n filepath
    - Display last n lines of the file
- alias c=clear
    - "c" is shortcut for "clear" command
    - c -- clear screen
    - alias
    - unalias c

IO Redirection

- Input redirection
    - command < filepath
    - Example: wc < in.txt
- Output redirection

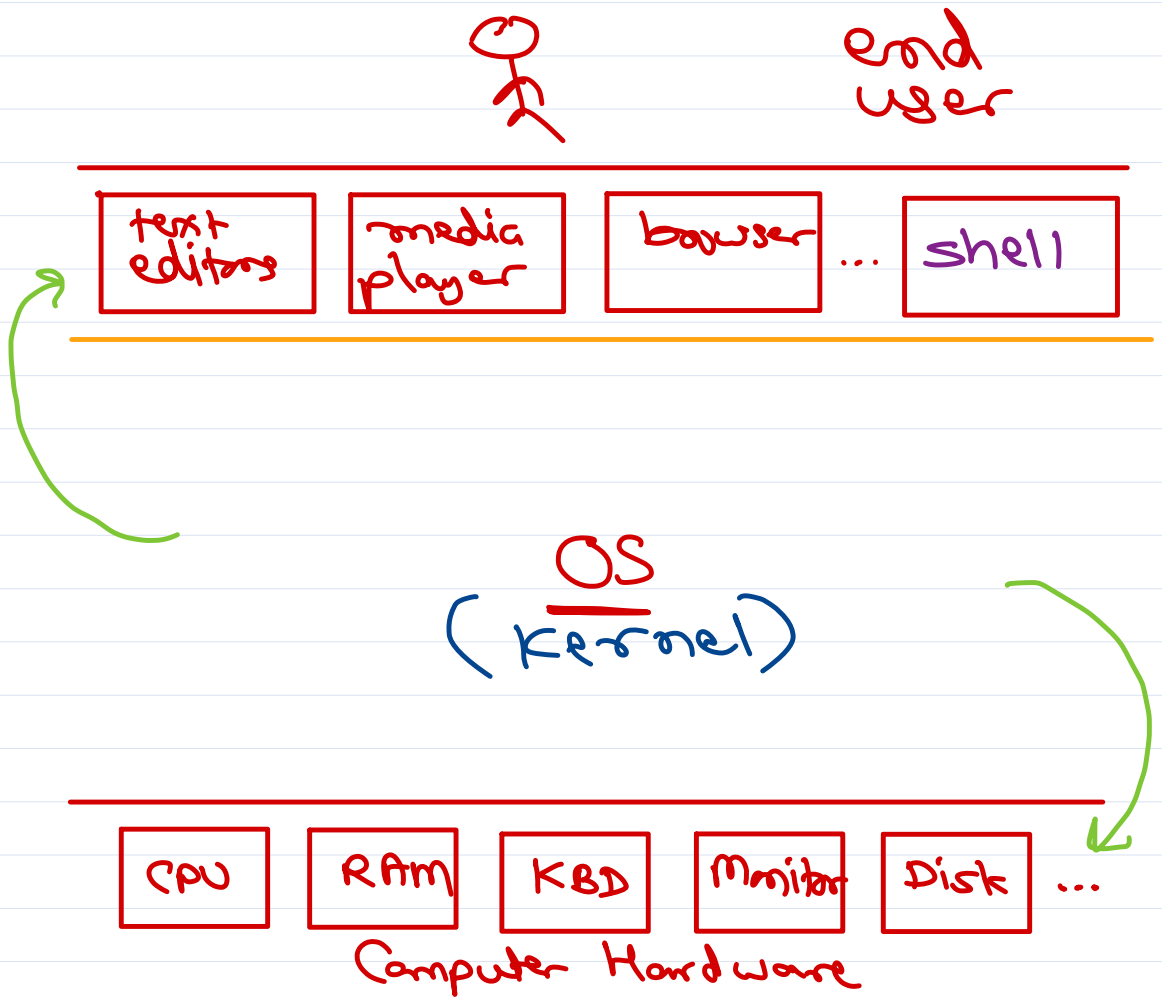
    - command > filepath
    - Example: history > out.txt
- Error redirection
    - command 2> filepath
    - Example: ls /abcd > err.txt
- Multiple redirection operators can be used in single command
    - Example: ls /home /abcd > out.txt 2> err.txt



Operating System Concepts

Sunbeam Infotech

What is OS?



1. end user → OS → hardware
2. applns → OS → hardware
3. Control program
4. resource manager
5. CD/DVD
   = Core OS
   + applications
   + utilities
6. Kernel = Core OS

OS functions

1. process mgmt
2. CPU scheduling
3. memory mgmt
4. file & io mgmt
5. hardware abstraction

must

6. networking
7. security & protection
8. user interfacing

optional

Process

```
int n1=123; <- data
int n2; <- bss
int main() {
    pf("Hello");
    return 0;
}
```

(sectioned binary)

a.out

- exe header
  - magic number
  - addr of entry pt
  - info of all sections
- text
- data | bss
- rodata
- Sym table

PE / ELF

loader

RAM

process

- stack
- heap
- data + bss
- rodata
- text

user space

kernel space

PCB

→ a.k.a. uarea (UNIX)

→ a.k.a. task_struct (Linux)

Sym table:

* info about vars & fns (global & static)
  - name
  - addr
  - size
  - section
  - flags

cmd > objdump -t a.out

PCB

1. pid
2. exit status
3. Sched info
   - state
   - time
   - priority
4. Files info
   - stdin (0)
   - stdout (1)
   - stderr (2)
   - ...
5. IPC info
6. mem info
   - base/limit
   - Page table
7. Kernel stack
8. Exec. ctx.

File

UNIX Philosophy: ① Files have spaces and Process have lives.
② Everything is file.

File = data + meta data
(contents) (information)
↓ ↓
data blocks file control block
a.k.a. inode (unix)

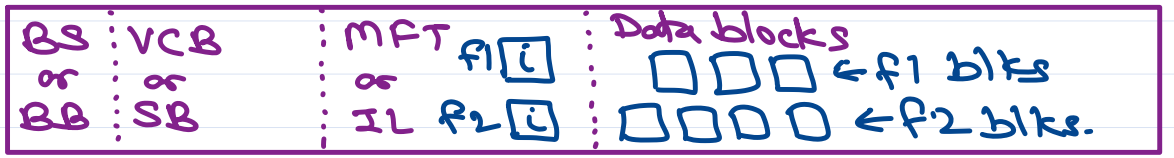
FCB/inode Contains

① type → file types
- – regular
- d directory
- l link
- p pipe
- s socket
- c char device
- b block device
  - → byte by byte data tx eg. kbd,..
  - → block by block data tx eg. storage devices,...
    - sector (512 bytes).

② name*
③ user/group+
④ permissions
⑤ size
⑥ links+
⑦ timestamps
⑧ info about data blocks

*not in unix
+ in unix

FileSystem: Way of organizing files on storage device



← disk partition →

① Boot block / Boot Sector:
- Bootstrap program, Bootloader,...

② Super Block / Volume Control Block:
- info of whole part i.e. label, blksize,
- total num of data block, num of free blocks and info of free blocks.

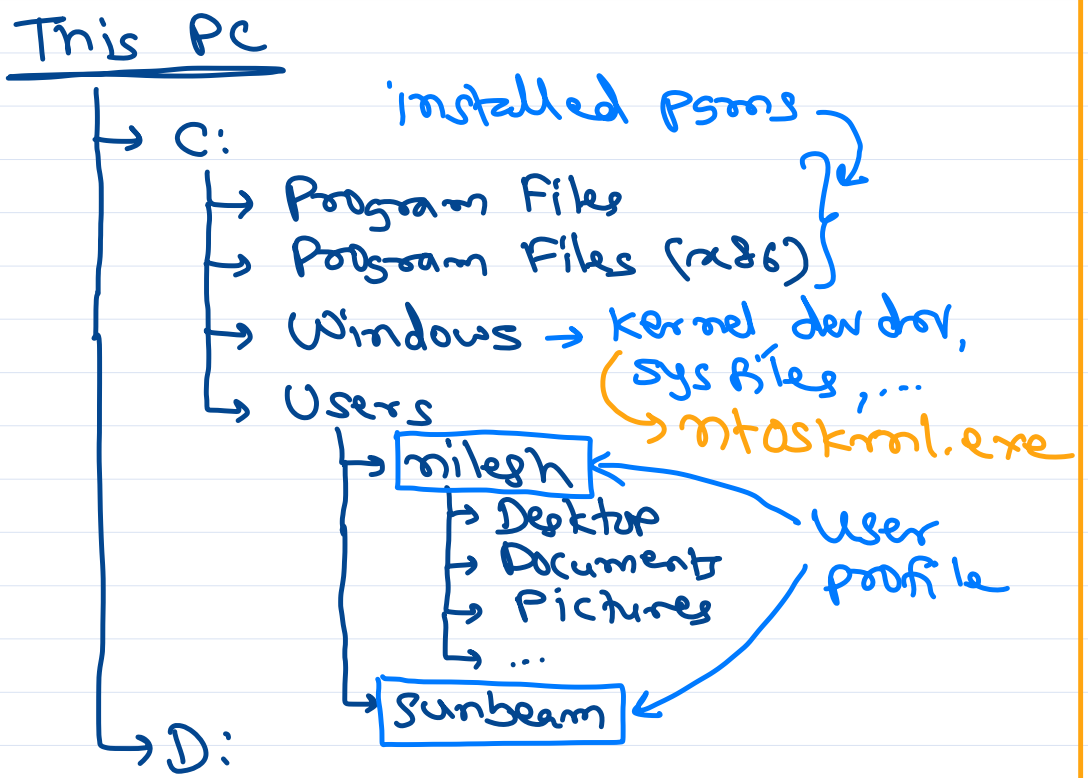
③ inode list / master file table:
- FCB/inodes of all files.

④ data blocks:
- data blocks of all files.

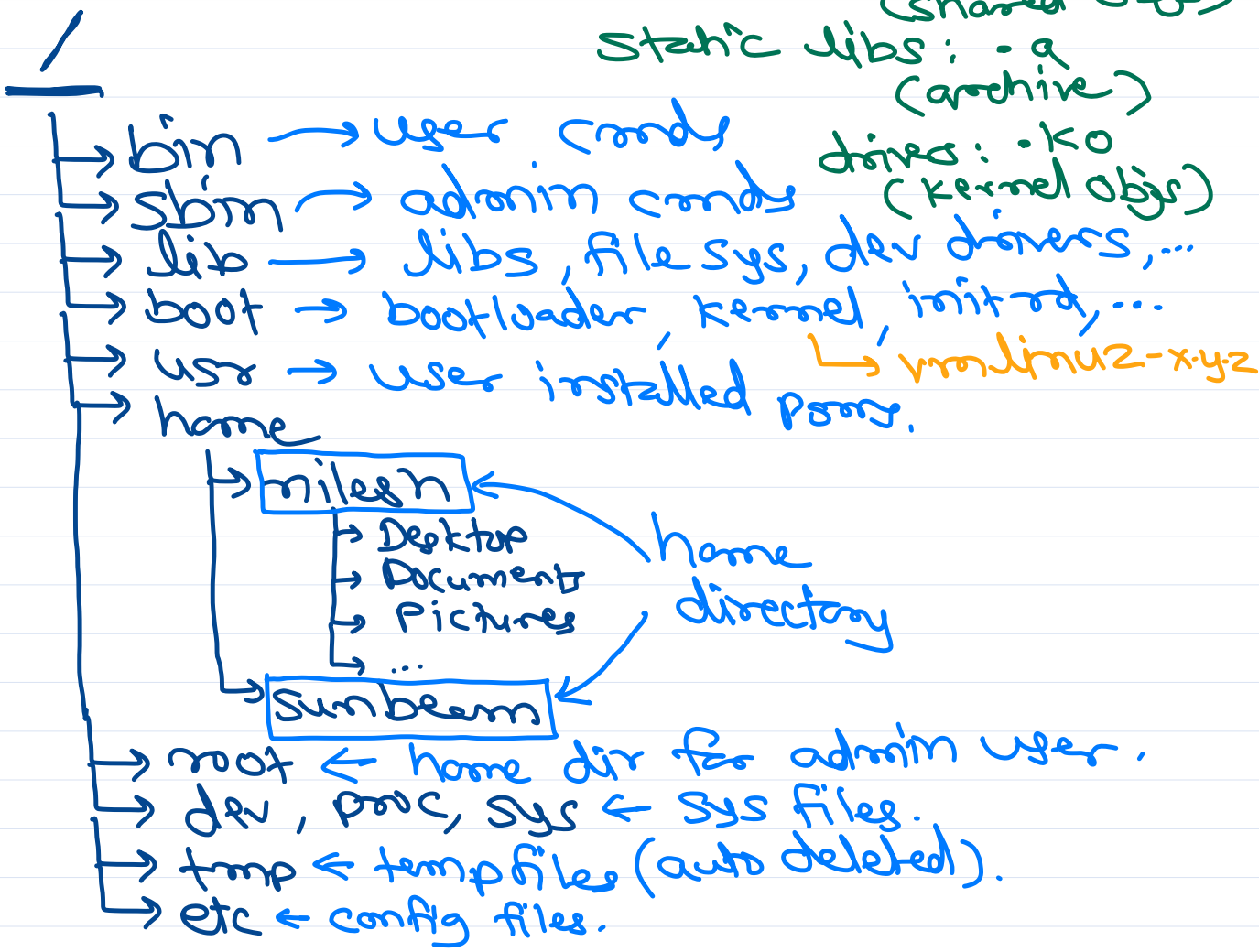
FS is created while format:
- DOS/Win → format cmd.
- Linux → mkfs cmd.

File System Hierarchy

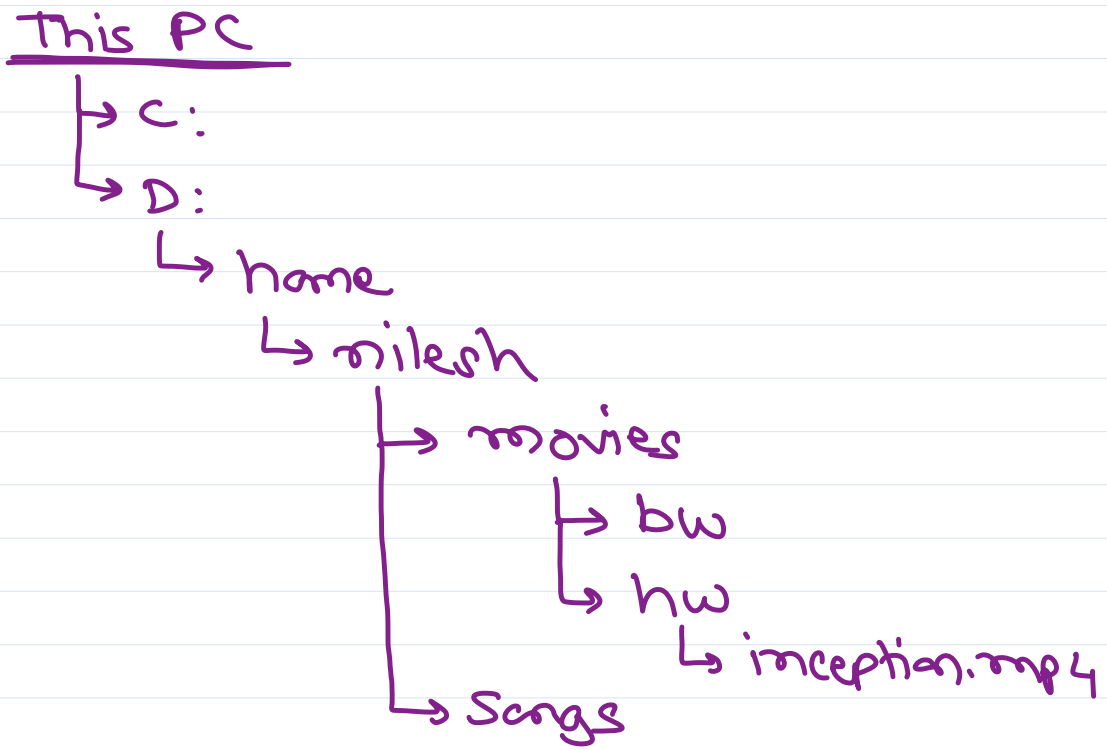
Windows file hierarchy



Linux file hierarchy



Absolute vs Relative path



This PC: D:\home\nilesh\movies\hw\inception.mp4 ← absolute path / full path start with drive letter.

nilesh: movies\hw\inception.mp4

hw: inception.mp4

songs: ..\movies\hw\inception.mp4

← relative path w.r.t. current directory

Absolute vs Relative path

/home/nilesh/movies/hw/inception.mp4 ← absolute path / full path start with /.

nilesh: movies/hw/inception.mp4

hw: inception.mp4

songs: ../movies/hw/inception.mp4

} ← relative path w.r.t. current directory

User interfacing

OS → UI is given by a special program.
called as "shell".
shell a.k.a. command interpreter.
end user → cmds → shell → kernel.

shell types:
① GUI → Graphical UI
② CLI/CUI → Commandline/ Console UI.

GNU (recursive acronym)
- UNIX
- Not
- GNU

gnu.org
- Linux
- GNOME
- cmds → grep, ...

Windows

- GUI → explorer.exe
- CLI → cmd.exe, powershell.exe

Linux

- GUI standards → GNOME, KDE, XFCE, ...
- CLI → bash, ksh, csh, zsh, ...

```
termind> echo $SHELL
```

Thank you!

Nilesh Ghule <nilesh@sunbeaminfo.com>

Operating System Concepts

Agenda

- OS Types
- Terminologies
  - Multi-Programming
  - Multi-Tasking
  - Multi-Threading
  - Multi-Processing
  - Multi-User
- Process Life Cycle
- CPU Scheduling
- Scheduling Algorithms
- Booting

Classification of OS

- OS can be categorized based on the target system (computers).
  - Mainframe systems
  - Desktop systems
  - Multi-processor (Parallel) systems
  - Distributed systems
  - Hand-held systems
  - Real-time systems

Mainframe systems

Resident Monitor

- Early (oldest) OS resides in memory and monitor execution of the programs. If it fails, error is reported.

- OS provides hardware interfacing that can be reused by all the programs.

Batch Systems

- The batch/group of similar programs is loaded in the computer, from which OS loads one program in the memory and execute it. The programs are executed one after another.
- In this case, if any process is performing IO, CPU will wait for that process and hence not utilized efficiently.

Multi-Programming

- In multi-programming systems, multiple program can be loaded in the memory.
- The number of program that can be loaded in the memory at the same time, is called as "degree of multi-programming".
- In these systems, if one of the process is performing IO, CPU can continue execution of another program. This will increase CPU utilization.
- Each process will spend some time for CPU computation (CPU burst) and some time for IO (IO burst).
    - If CPU burst > IO burst, then process is called as "CPU bound".
    - If IO burst > CPU burst, then process is called as "IO bound".
- To efficiently utilize CPU, a good mix of CPU bound and IO bound processes should be loaded into memory. This task is performed by an unit of OS called as "Job scheduler" OR "Long term scheduler".
- If multiple programs are loaded into the RAM by job scheduler, then one of process need to be executed (dispatched) on the CPU. This selection is done by another unit of OS called as "CPU scheduler" OR "Short term scheduler".

Multi-tasking OR time-sharing

- CPU time is shared amoung multiple processes in the main memory is called as "multi-tasking".
- In such system, a small amount of CPU time is given to each process repeatedly, so that response time for any process < 1 sec.
- With this mechanism, multiple tasks (ready for execution) can execute concurrently.
- There are two types of multi-tasking:
    - Process based multitasking: Multiple independent processes are executing concurrently. Processes running on multiple processors called as "multi-processing".
    - Thread based multi-tasking OR multi-threading: Multiple parts/functions in a process are executing concurrently.

Multi-user

- Multiple users can execute multiple tasks concurrently on the same systems. e.g. IBM 360, UNIX, Windows Servers, etc.
- Each user can access system via different terminal.
- There are many UNIX commands to track users and terminals.
    - tty, who, who am i, whoami, w

Desktop systems

- Personal computers -- desktop and laptops
- User convinence and Responsiveness
- Examples: Windows, Mac, Linux, few UNIX, ...

Multiprocessor systems

- The systems in which multiple processors are connected in a close circuit is called as "multiprocessor computer".
- The programs/OS take advantage of multiple processors in the computer are called as "Multiprocssing" programs/OS.
    - Windows Vista: First Windows OS designed for multi-processing.
    - Linux 2.5+: Linux started supporting multi-processing.
- Modern PC architectures are multi-core arch i.e. multiple CPUs on single chip.
- Since multiple tasks can be exeuted on these processors simultaneously, such systems are also called as "parallel systems".
- Parallel systems have more throughput (Number of tasks done in unit time).
- There are two types of multiprocessor systems:
    - Asymmetric Multi-processing
    - Symmetric Multi-processing

Asymmetric Multi-processing

- OS treats one of the processor as master processor and schedule task for it. The task is in turn divided into smaller tasks and get them done from other processors.

Symmetric Multi-processing

- OS considers all processors at same level and schedule tasks on each processor individually.
- All modern desktop systems are SMP.

Distributed systems

- Multiple computers connected together in a close network is called as "distributed system".
- Its advantages are high availability (24x7), high scalability (many clients, huge data), fault tolerance (any computer may fail).
- The requests are redirected to the computer having less load using "load balancing" techniques.
- The set of computers connected together for a certain task is called as "cluster". Examples: Linux.

Handheld systems

- OS installed on handheld devices like mobiles, PDAs, iPODs, etc.
- Challenges:
  - Small screen size
  - Low end processors
  - Less RAM size
  - Battery powered
- Examples: Symbian, iOS, Linux, PalmOS, WindowsCE, etc.

Realtime systems

- The OS in which accuracy of results depends on accuracy of the computation as well as time duration in which results are produced, is called as "RTOS".
- If results are not produced within certain time (deadline), catastrophic effects may occur.
- These OS ensure that tasks will be completed in a definite time duration.
- Time from the arrival of interrupt till begin handling of the interrupt is called as "Interrupt Latency".
- RTOS have very small and fixed interrupt latencies.
- RTOS Examples: uC-OS, VxWorks, pSOS, RTLinux, FreeRTOS, etc.

Process Life Cycle

Process States

- New
  - New process PCB is created and added into job queue. PCB is initialized and process get ready for execution.

- Ready
  - The ready process is added into the ready queue. Scheduler pick a process for scheduling from ready queue and dispatch it on CPU.
- Running
  - The process runs on CPU. If process keeps running on CPU, the timer interrupt is used to forcibly put it into ready state and allocate CPU time to other process.
- Waiting
  - If running process request for IO device, the process waits for completion of the IO. The waiting state is also called as sleeping or blocked state.
- Terminated
  - If running process exits, it is terminated.
- Linux: TASK_READY/TASK_RUNNING (R), TASK_INTERRUPTIBLE (S), TASK_UNINTERRUPTIBLE (D), TASK_STOPPED(T), TASK_ZOMBIE (Z), TASK_DEAD (X)

Types of Scheduling

Non-preemptive

- The current process gives up CPU voluntarily (for IO, terminate or yield).
- Then CPU scheduler picks next process for the execution.
- If each process yields CPU so that other process can get CPU for the execution, it is referred as "Co-operative scheduling".

Preemptive

- The current process may give up CPU voluntarily or paused forcibly (for high priority process or upon completion of its time quantum)

Scheduling criterias

CPU utilization: Ideal - max

- On server systems, CPU utilization should be more than 90%.

- On desktop systems, CPU utilization should around 70%.

Throughput: Ideal - max

- The amount of work done in unit time.

Waiting time: Ideal - min

- Time spent by the process in the ready queue to get scheduled on the CPU.
- If waiting time is more (not getting CPU time for execution) -- Starvation.

Turn-around time: Ideal - CPU burst + IO burst

- Time from arrival of the process till completion of the process.
- CPU burst + IO burst + (CPU) Waiting time + IO Waiting time

Response time: Ideal - min

- Time from arrival of process (in ready queue) till allocated CPU for first time.

Scheduling Algorithms

FCFS

- Process added first in ready queue should be scheduled first.
- Non-preemptive scheduling
- Scheduler is invoked when process is terminated, blocked or gives up CPU is ready for execution.
- Convoy Effect: Larger processes slow down execution of other processes.

SJF

- Process with lowest burst time is scheduled first.
- Non-preemptive scheduling
- Minimum waiting time

SRTF - Shortest Remaining Time First

- Similar to SJF - but Preemptive scheduling
- Minimum waiting time

Priority

- Each process is associated with some priority level. Usually lower the number, higher is the priority.
- Preemptive scheduling or Non Preemptive scheduling
- Starvation
    - Problem may arise in priority scheduling.
    - Process not getting CPU time due to other high priority processes.
    - Process is in ready state (ready queue).
    - May be handled with aging -- dynamically increasing priority of the process.

Round-Robin

- Preemptive scheduling
- Process is assigned a time quantum/slice.
- Once time slice is completed/expired, then process is forcibly preempted and other process is scheduled.
- Min response time.

Fair-share

- CPU time is divided into epoch times.
- Each ready process gets some time share in each epoch time.
- Process is assigned a time share in proportion with its priority.
- In Linux, processes with time-sharing (TS) class have nice value. Range of nice value is -20 (highest priority) to +19 (lowest priority).
- terminal> ps -A -o pid,cls,ni,cmd

Linux scheduling classes

- Linux supports real-time (soft) and non-realtime scheduling.

- For real-time scheduling classes, high priority process is always scheduled before low priority process. In other words, high priority process never waits for a low priority process.
- Real-time scheduling classes
  - SCHED_FIFO
  - SCHED_RR
- Non Real-time scheduling classes
  - SCHED_OTHER or SCHED_NORMAL
  - SCHED_BATCH
  - SCHED_IDLE

Linux

Linux commands

- ps command
  - ps -- displays processes running in current terminal
  - ps -A -- displays all processes
  - ps aux -- displays all processes (formatted)
- kill command -- send signal to process
  - kill -sig pid
  - e.g. kill -9 7893
  - kill -l --> list of all signals
    - 9 - SIGKILL
    - 2 - SIGINT (Ctrl+C)
    - 19 - SIGSTOP (Ctrl+S)
    - 18 - SIGCONT (Ctrl+Q)
    - 15 - SIGTERM
  - pkill -sig program-name --> kill all processes of given program
    - pkill -9 java
    - pkill -kill chrome
  - When signal is sent to process, it will cause some default action.
    - Term: Terminate process (SIGINT, SIGKILL, SIGHUP, SIGALRM, ...)

        - Core: Abort process (SIGSEGV)
        - Stop: Suspend process (SIGSTOP)
        - Cont: Resume process (SIGCONT)
        - Ign: Ignore signal (SIGCHLD)
    - A program may handle the signals to implement other desired actions.
    - SIGKILL and SIGSTOP signals cannot be handled by the process.
- tr
- chmod
- chown
- ln

Multi-user related commands

- tty command
    - Display current terminal name
- who command
    - Display all users logged into the system
- w command
    - Display all users logged into the system (more details)
- whoami command
    - Display current user name
- "who am i" command
    - Display current user name and terminal.

grep

- Find a pattern in text file(s).
- Regular expressions are patterns used to match character combinations in strings.
- A regular expression pattern is composed of simple characters, or a combination of simple and special characters e.g. /abc/, /ab*c/
- Pattern is given using regex wild-card characters.
    - Basic wild-card characters
        - $ - find at the end of line.

        - ^ - find at the start of line.
        - [ ] - any single char in give range or set of chars
        - [^] - any single char not in give range or set of chars
        - . - any single character
        - zero or more occurrences of previous character
    - Extended wild-card characters
        - ? - zero or one occurrence of previous character
        - one or more occurrences of previous character
        - {n} - n occurrences of previous character
        - {,n} - max n occurrences of previous character
        - {m,} - min m occurrences of previous character
        - {m,n} - min m and max n occurrences of previous character
        - () - grouping (chars)
        - (|) - find one of the group of characters
- Regex commands
    - grep - GNU Regular Expression Parser - Basic wild-card
    - egrep - Extended Grep - Basic + Extended wild-card
    - fgrep - Fixed Grep - No wild-card
- Command syntax
    - grep "pattern" filepath
    - grep [options] "pattern" filepath
        - -c : count number of occurrences
        - -v : invert the find output
        - -i : case insensitive search
        - -w : search whole words only
        - -R : search recursively in a directory
        - -n : show line number.

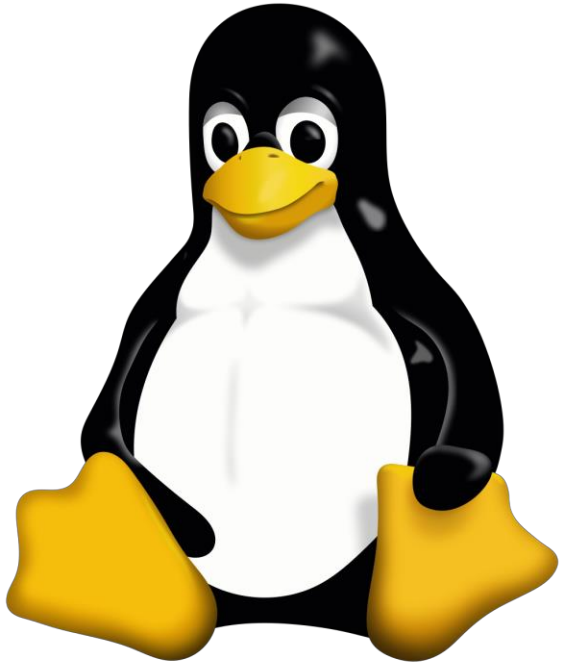
Redirection

- By default every process opens 3 file descriptors
    - fd = 0 -> standard input to a program

    - fd = 1 -> standard output to a program
    - fd = 2 -> standard error to a program
- You can redirect each of these independently.
- According to direction of redirection, there are three types
- Input redirection
    - “<” is used for input redirection
- Output redirection
    - “>” is used for output redirection
- Error redirection
    - “2>” is used for error redirection

Pipe

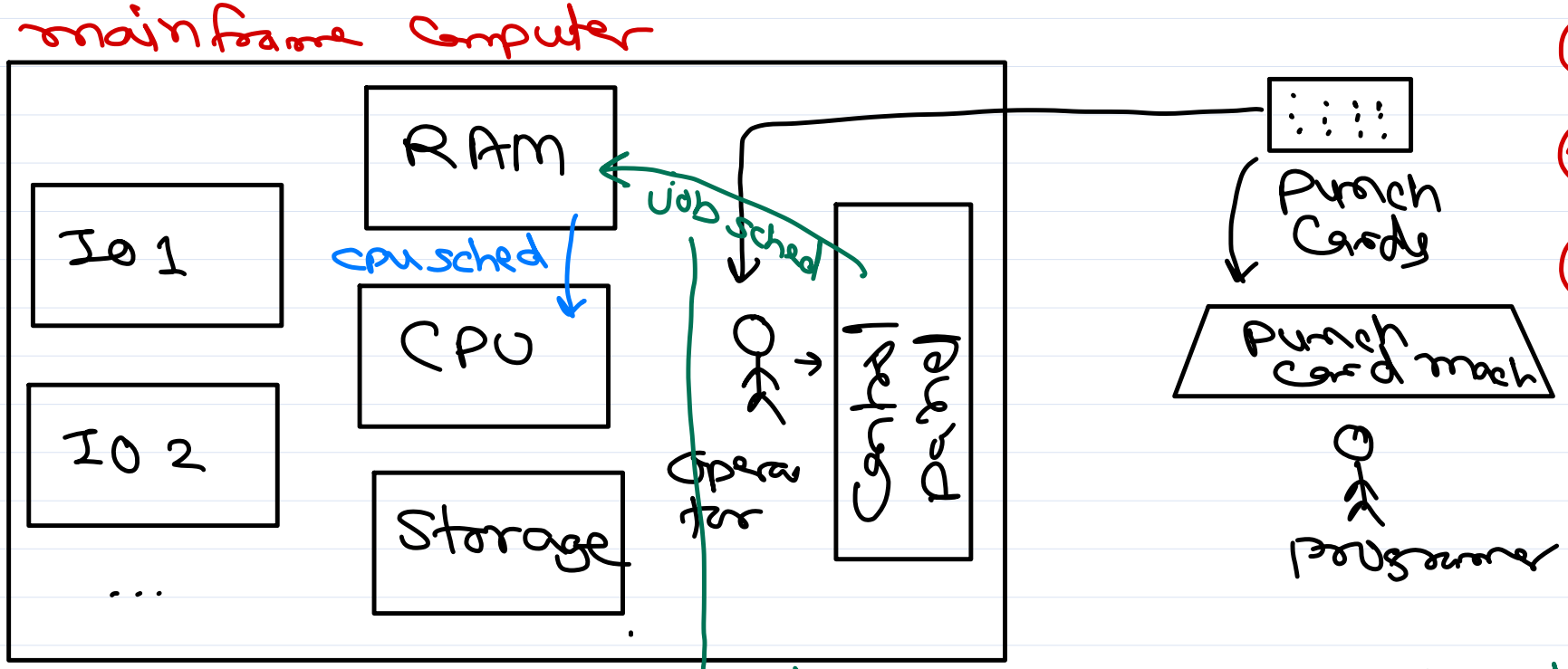
- Using pipe, we can redirect output of any command to the input of any other command.
- Two processes are connected using pipe operator (|).
- Two processes runs simultaneously and are automatically rescheduled as data flows between them.
- If you don’t use pipes, you must use several steps to do single task.
- Syntax: command1 | command2
- E.g.
    - who | wc



Operating System Concepts

Sunbeam Infotech

Mainframe systems



① Resident monitor

② Batch System

③ Multi-programming
- Loading multiple programs in main memory
- Aim: Better CPU utilization
- Degree of multi-prog: num of progs that can be loaded in mem

program exec time
→ CPU burst time
→ IO burst time

IO bound process: IO time > CPU time
CPU bound process: CPU time > IO time

→ decides programs to be loaded in RAM.
Combn of IO bound & CPU bound jobs is for better CPU & IO utilization.
Modern OS don't have job sched.

④ multi-tasking / time sharing.

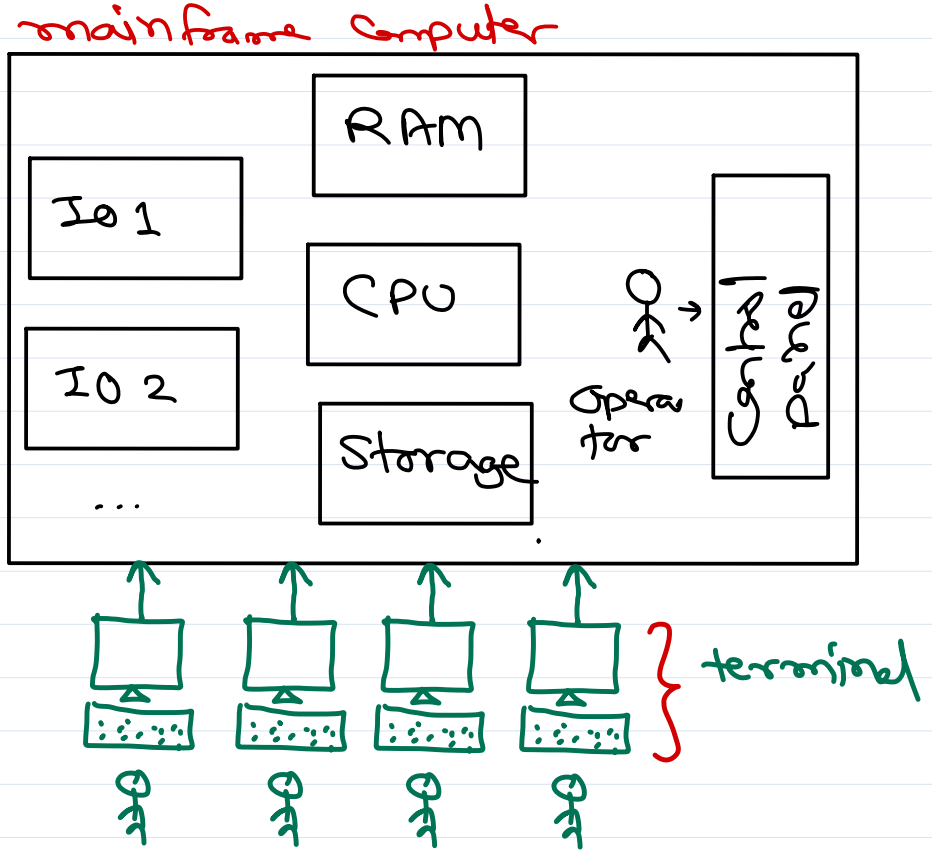
Mainframe systems

④ Multi-tasking (Time-sharing)

- Sharing CPU time among multiple programs present in main memory & ready for execution.
  - * Process based multi-tasking
    - Share CPU time in independent processes.
  - * Thread based multi-tasking
    - share CPU time in multiple threads of same/diff processes.
    - Threads are created to do multiple tasks concurrently within a single process.
    - a.k.a. multi-threading.

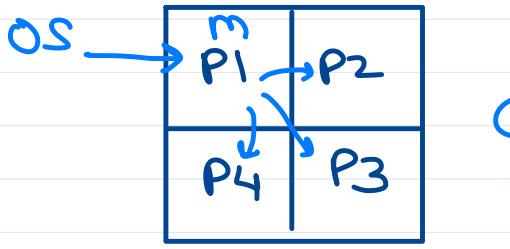
⑤ multi-user:

- multiple users execute multiple programs concurrently on same computer.

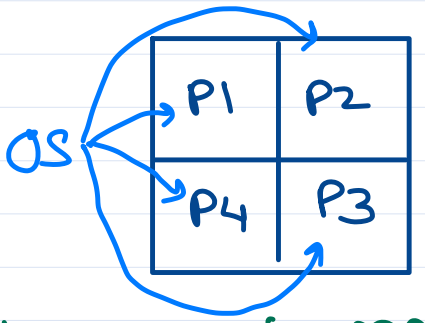


OS types

1. mainframe systems
   e.g. UNIX, IBM360, ...
2. desktop systems
   e.g. Linux, Mac, Windows
3. parallel systems
   - Using multiple CPUs in same computer to perform tasks.

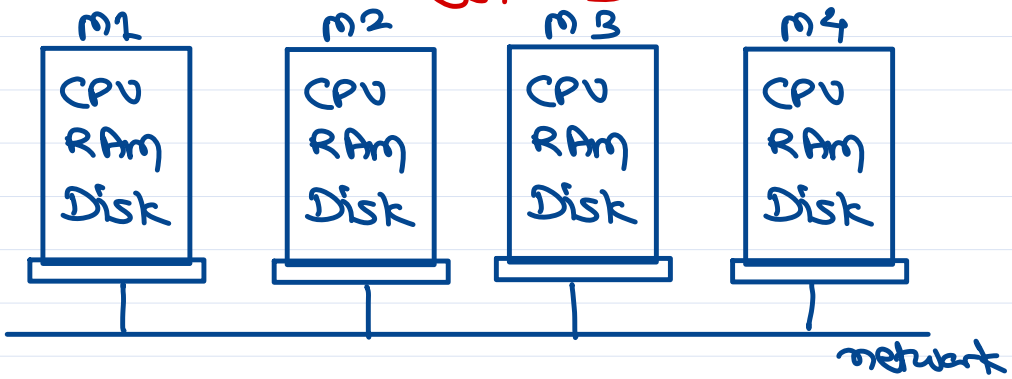


asymmetric MP.



symmetric MP.
e.g. Linux, Windows Vista.

4. Distributed systems



cluster: set of computers connected in a network for a dedicated purpose

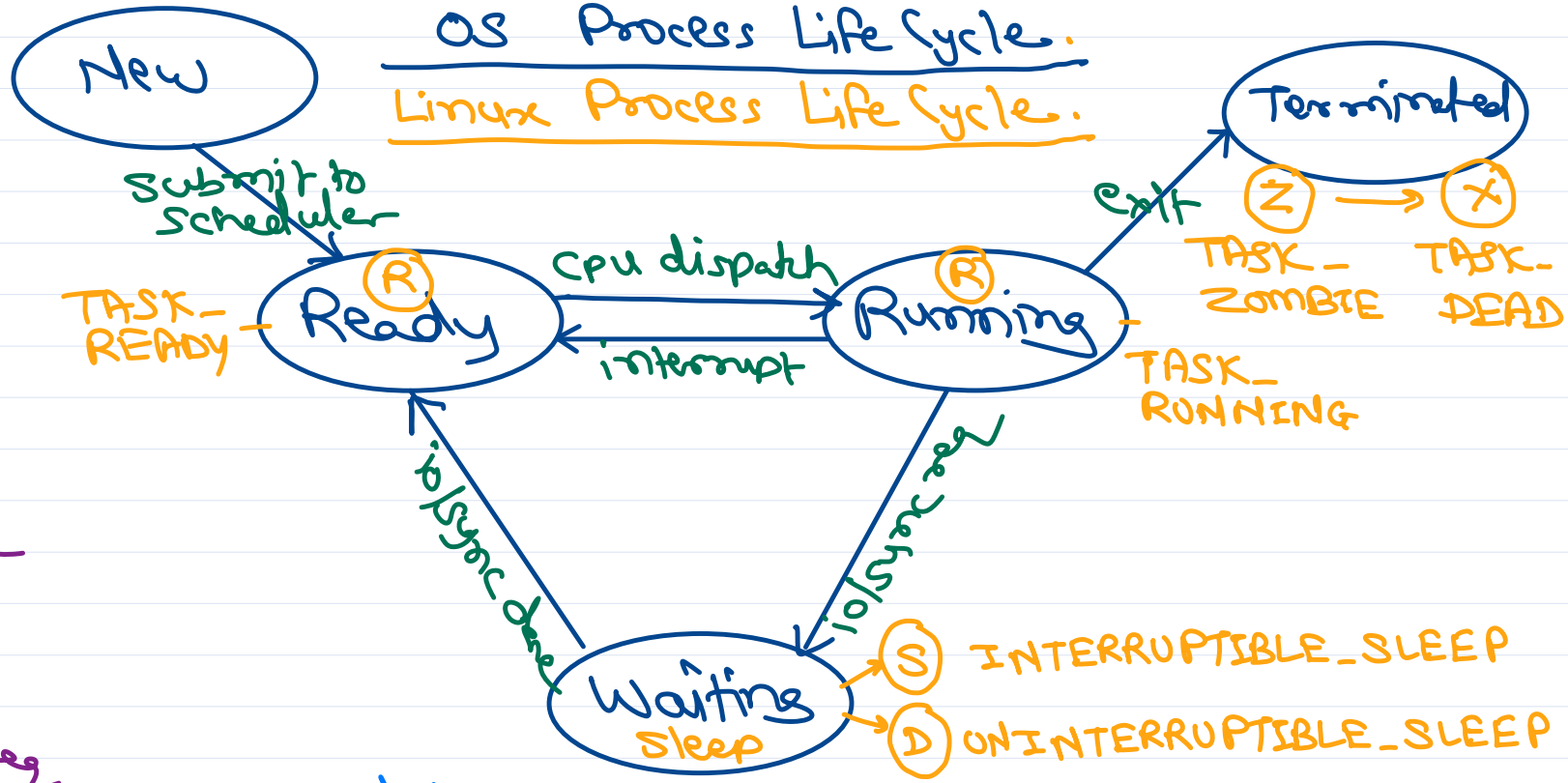
5. hand held systems
   e.g. Symbian, Android, iOS, ...
6. real time system
   accuracy of result not only depends on correctness of calculation but also depend on time in which result is produced.

   e.g. Win CE, FreeRTOS, uCOS, Xenomai, RTAI, uITron, pSos, VxWorks, ...

Process Life Cycle

OS data structures

1. job queue / process list
   - list of all processes (PCB)
2. ready queue / runqueue
   - list of processes ready for execution on CPU.
   - CPU scheduler pick up process from ready queue and then dispatcher load it into CPU.
3. waiting queues
   - processes doing IO/sync ops are added into wait queue of respective IO devices / sync objs.



CPU scheduler called

1. Running → Terminated
2. Running → Waiting

} non-preemptive scheduling aka. cooperative scheduling

3. Running → Ready
4. Waiting → Ready

} preemptive scheduling

CPU scheduling

Scheduling Criterias

1. CPU utilization ↑
2. waiting time ↓
3. turn around time ↓
   min = CPU time + Io time
4. response time ↓
5. throughput ↑

Scheduling algorithms

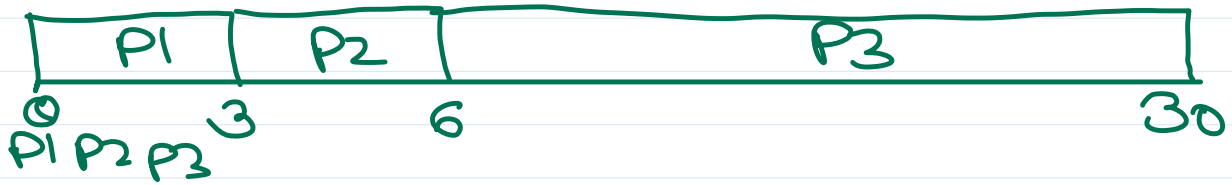
1. FCFS
2. SJF
3. Priority
4. RR
5. Fair share

FCFS (non-preemptive)

Process	time	wait	TAT
P1	3	0	3
P2	3	3	6
P3	24	6	30

avg wait $= \frac{0+3+6}{3} = 3$

avg TAT $= \frac{3+6+30}{3} = 13$



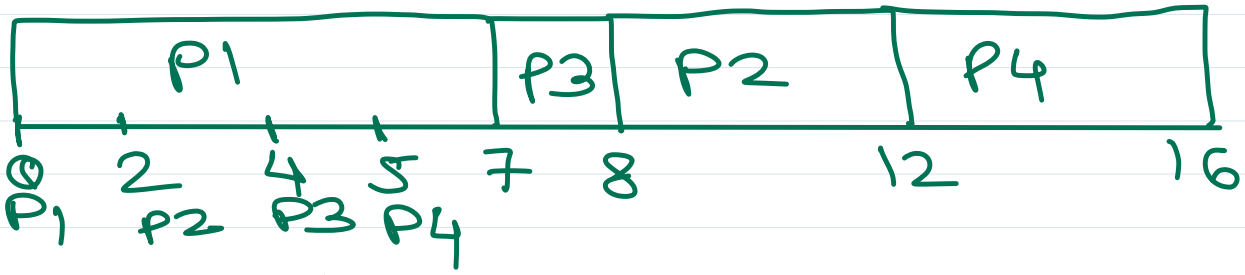
Process	time	wait	TAT
P1	24	0	
P2	3	24	
P3	3	27	

avg wait = 17.

Convoy effect

SJF (non-preemptive)

process	arrival time	wait	
P1	0	7	0
P2	2	4	6
P3	4	1	3
P4	5	4	7

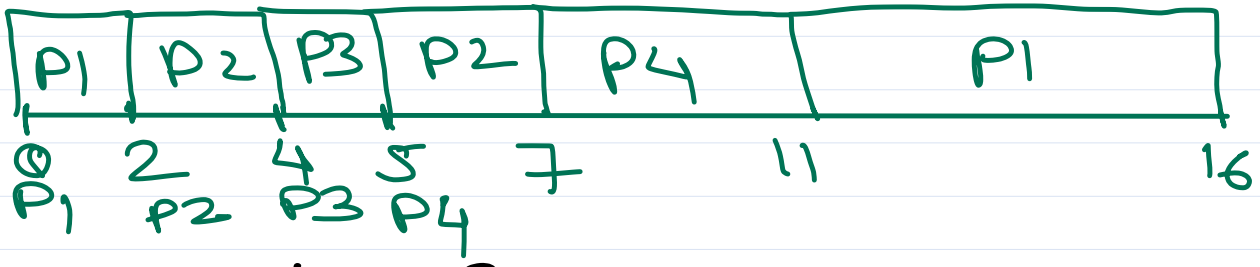


avg wait = 4 ms.

SJF (preemptive) = SRTF

process	arrival time	wait	
P1	0	7-5x	9
P2	2	4-2x	1
P3	4	1-1x	0
P4	5	4-4x	2

* gives min avg wait time.



avg wait = 3 ms

CPU scheduling

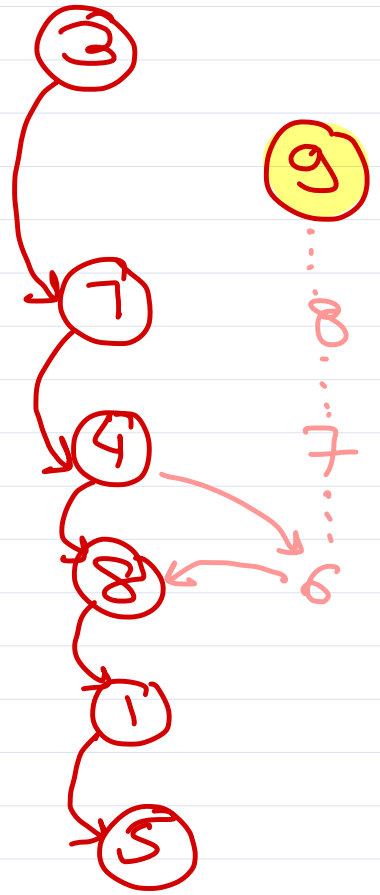
Priority Sched

- non-preemptive
- preemptive

each process is associated with a number called as "priority".
Usually, lower number indicate higher priority.

due to high priority processes, a low priority process may not get enough CPU time for execution
⇒ Starvation.

increase priority of starved processes periodically so that it will get cpu time for execution sooner ⇒ aging.



CPU scheduling

RR (preemptive)

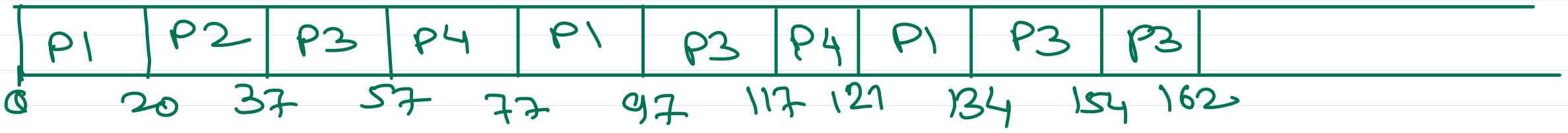
assign fixed time slice/ time quantum to each process repeatedly.

process	time	rem	wait	resp
P1	53	~~33 13~~ X	81	0
P2	17	X	20	20
P3	68	~~48 28 8~~ X	94	37
P4	24	~~4~~ X	97	57

avg wait = 73

min resp time.

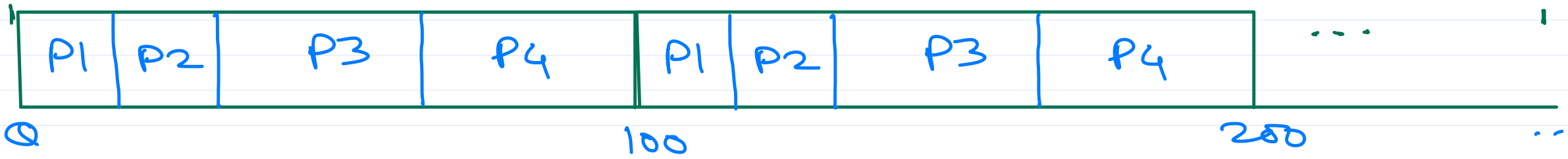
Time quantum = 20



CPU scheduling

Fair share also

epoch time = 100 ms



process	priority
P1	10 (L)
P2	10 (L)
P3	5 (H)
P4	5 (H)

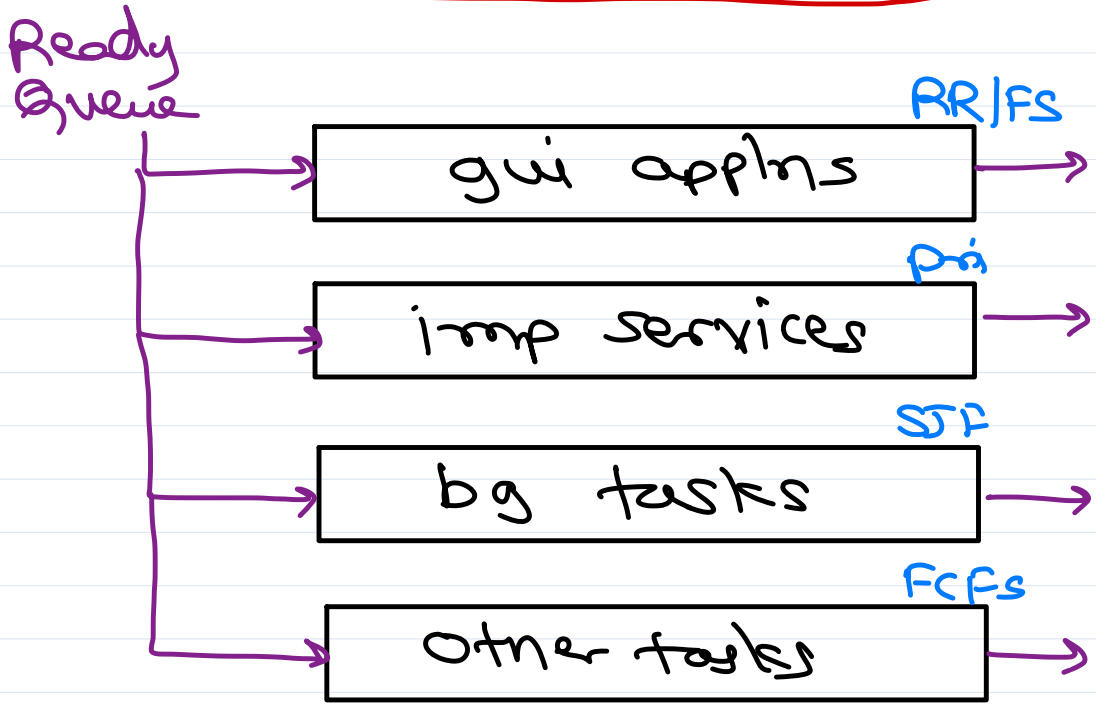
Called as "nice" value in Linux.

CPU time divided into epoch times and in each epoch CPU time share is given to each ready process based its priority.

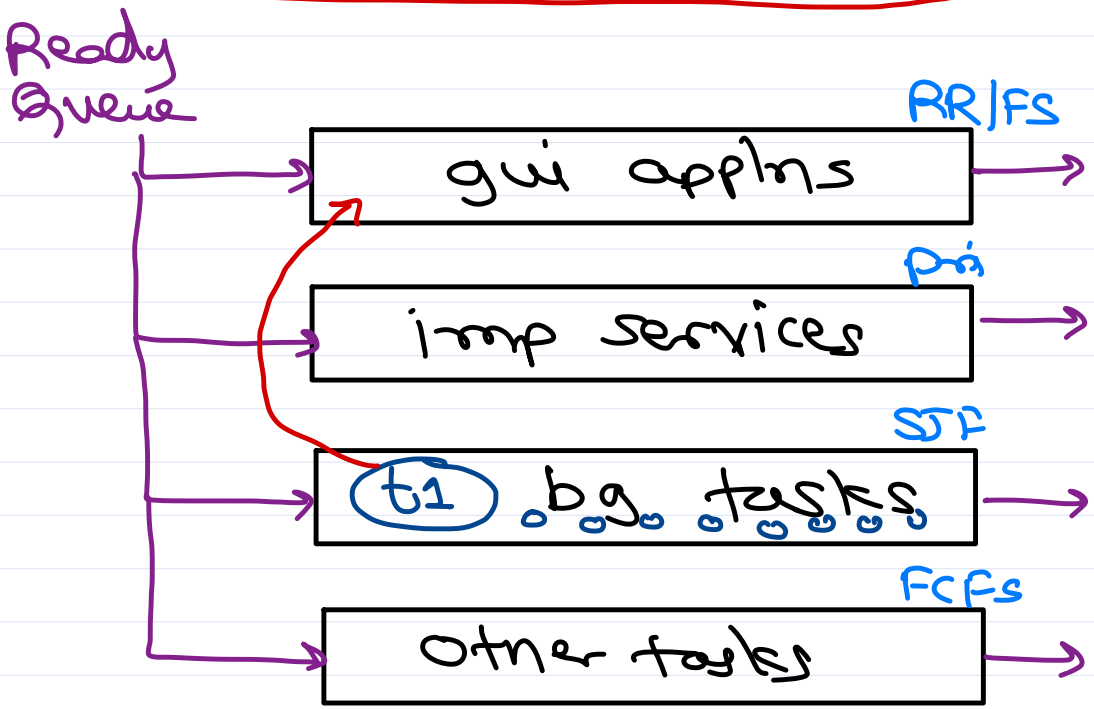
High priority process gets more CPU time w.r.t. low priority process.

CPU scheduling

multi-level sched queue



multi-level feedback queue



Linux sched classes

* Realtime sched
  - (A) SCHED_FIFO
  - (B) SCHED_RR
* Non-realtime sched
  - (C) SCHED_OTHER/NORMAL (fair share)
  - (D) SCHED_BATCH
  - (E) SCHED_IDLE

VI editor

- text editor in CLI
- developed by BSD UNIX
  - UCB - Bill Joy
- VI 'improved' - vim

cmd> vim filepath

- VI editor modes
  - Command mode (default).
    - press esc.
  - insert/edit mode.
    - press "i"

Commands

1. save/write → :w
2. quit → :q
3. save & quit → :wq
4. copy cur line → yy
   copy n lines → nyy
   copy line x to line z → :x,zy
   copy after cursor → y$
   copy before cursor → y^
   copy cur word → yw
5. cut (same as copy) → (y) → (d)
6. paste → p
7. undo → u
8. redo → ctrl + R
9. run linux cmd → :! command

Thank you!

Nilesh Ghule <nilesh@sunbeaminfo.com>

Operating System Concepts

Agenda

- VIM editor
  - ~/.vimrc file
- Shell scripts
  - float calculations
  - if-else statement
  - Relational and logical operators
  - File operators
  - case statement
  - loop statements
  - array
  - string operations
  - functions
  - positional params
- Booting
- Dual mode protection
- System calls
- Kernel types

Shell scripts

- Refer codes

Computer structure

- CPU: Genral purpose processor for program/OS execution
- Memory: RAM
- Storage: Disk

- IO: Keyboard, Monitor
- Connected by "bus".
- Each IO device has a "dedicated" "internal" processing unit -- IO device controller.

Computer IO (Input Output)

- Synchronous IO: CPU is waiting for IO to complete.
  - Hw technique: Polling
- Asynchronous IO: CPU is not waiting for IO to complete (doing some other task)
  - Hw technique: Interrupts
  - OS maintains a device status table to keep track of IO devices (busy/idle) and processes waiting for those IO devices.

Interrupt Processing

- IO event is sensed by IO device controllers.
- It will be conveyed to CPU as a special signal - Interrupt.
- CPU pause current execution and execute interrupt handler.
- "Interrupt handler" will get address of "ISR" (from IVT) and execute ISR.
- When ISR is completed, execution resumes where it was paused.

Hardware vs Software interrupt

- Hardware -- interrupts from hardware peripherals.
- Software interrupt
  - Special instructions (Assembly/Machine level) when executed, current execution is paused, interrupt handler is executed and then the paused execution resumes.
  - Arch specific:
    - 8085/86: INT
    - ARM 7: SWI
    - ARM Cortex: SVC
  - Also called as "Trap" in few architecture.

Interrupt Controller

- Convey the interrupts from various peripherals to the CPU.
- Also manage priority of the interrupt (when multiple interrupts arrives at same time).
- e.g. 8085/86 <-- 8259, Modern x86 processors (apic), ARM-7 (VIC), ARM-CM3 (NVIC), ...

System Calls

- Software interrupt is used to implement OS/Kernel services.
- Functions exposed by the kernel so that user programs can access kernel functionalities, are called as "System calls".
  - e.g. Process Mgmt: create process, exit process, communication, synchronization, etc.
  - e.g. File Mgmt: create file, write file, read file, close file, etc.
  - e.g. Memory Mgmt: alloc memory, release memory, etc.
  - e.g. CPU Scheduling: Change process priroty, change process CPU affinity, etc.
- System calls are specific to the OS:
  - UNIX: 64 syscalls e.g. fork(), ..
  - Linux: 300+ syscalls e.g. fork(), clone(), ...
  - Windows: 3000+ syscalls e.g. CreateProcess(), ...

Types of kernel

Monolithic Kernel

- Multiple kernel source files are compiled into single kernel binary image. Such kernels are "monolithic" kernels.
- Since all functionalities present in single binary image, execution is faster.
- If any functionality fails at runtime, entire kernel may crash.
- Any modification in any component of OS, needs recompilation of the entire OS.
- Examples: BSD Unix, Windows (ntoskrnl.exe), Linux (vmlinuz), etc.

Micro-kernel

- Kernel is having minimal functionalities and remaining functionalities are implemented as independent processes called as "servers".
  - e.g. File management is done by a program called as "file server".
- These servers communicate with each other using IPC mechanism (message passing) and hence execution is little slower.
- If any component fails at runtime, only that process is terminated and rest kernel may keep functioning.

- Any modification in any component need to recompile only that component.
- Examples: Symbian, MACH, etc.

Modular Kernel

- Dynamically loadable modules (e.g. .dll / .so files) are loaded into calling process at runtime.
- In modular systems, kernel has minimal functionalities and rest of the functionalities are implemented as dynamically loadable modules.
- These modules get loaded into the kernel whenever they are called.
- As single kernel process is running, no need of IPC for the execution and thus improves performance of the system.
- Examples: Windows, Linux, etc.

Hybrid Kernel

- Mac OS X kernel is made by combination of two different kernels.
- BSD UNIX + MACH = Darwin

Linux - OS Structure

Linux components

- Linux kernel has static and dynamic components.
- Static components are
  - Scheduler
  - Process management
  - Memory management
  - IO subsystem (core)
  - System calls
- Dynamic components are
  - File systems (like ext3, ext4, FAT)
  - Device drivers
- Static components are compiled into the kernel binary image.
  - They are kernel components.
  - The kernel image is /boot/vmlinuz.

- Dynamic components are compiled into kernel objects (*.ko files).
  - They are non-kernel components.
  - They are located in /lib/modules/kernel-version.

Booting

- Process from computer power on to OS startup.

Bootable device

- Stoarage device (disk, pen drive, CD/DVD, etc.) whose boot block contains bootstrap program, is said to be Bootable device.

Bootstrap program

```
* Bootstrap program can load OS kernel into RAM and start its execution.
* Bootstrap program is different for each OS each version.
* Bootstrap is located in first sector (512 bytes) of a disk/partition.
```

Bootloader program

- Bootloader program can be configured to show multiple boot options to end user.
- Depending on user selection, it runs Bootstrap program of corresponding OS.
- There are many important bootloader programs.
  - ntldr (Before Windows Vista) --> boot.ini
  - "bootmgr" (Windows Vista Onwards) --> bcd (boot config data)
    - (admin command prompt) cmd> bcdedit
  - LiLo --> Linux Loader
  - "GrUB" (Grand Unified Boot Loader) --> menu.lst or grub.cfg
  - BTX (BooT eXtended) --> BSD Unix
  - SILo (Sparc Interactive Loader) --> Solaris
  - Bootcamp --> Mac OS X

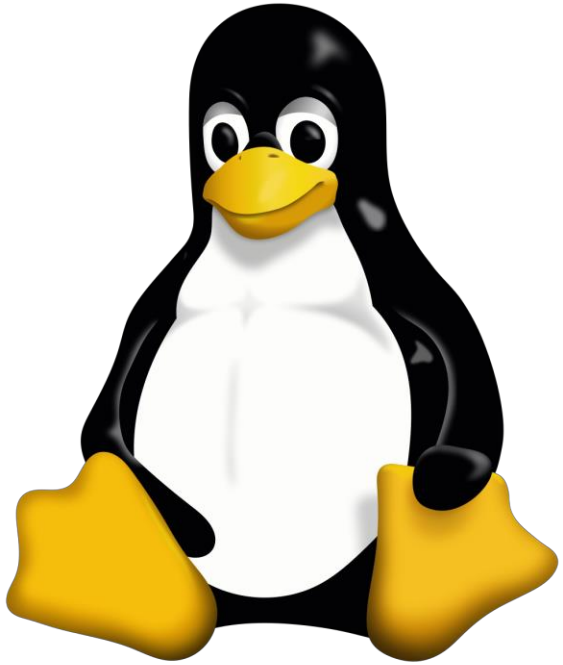
    - "uBoot" --> Linux bootloader for Embedded
- The Bootloader has some config file associated with it.

Bootstrap loader

- The set of programs fixed in Base ROM (Motherboard) is called as "Firmware".
    - BIOS (Basic Input Output System) -- Firmware developed for PC by IBM + MS.
    - EFI (Extensible Firmware Interface) -- Firmware developed for PC by Intel + HP.
- Bootstrap loader is a program from Base ROM.
- It find the bootable device as per "boot device priority" in the BIOS setup.
- Once bootable device is found, it starts its bootloader.

Booting steps

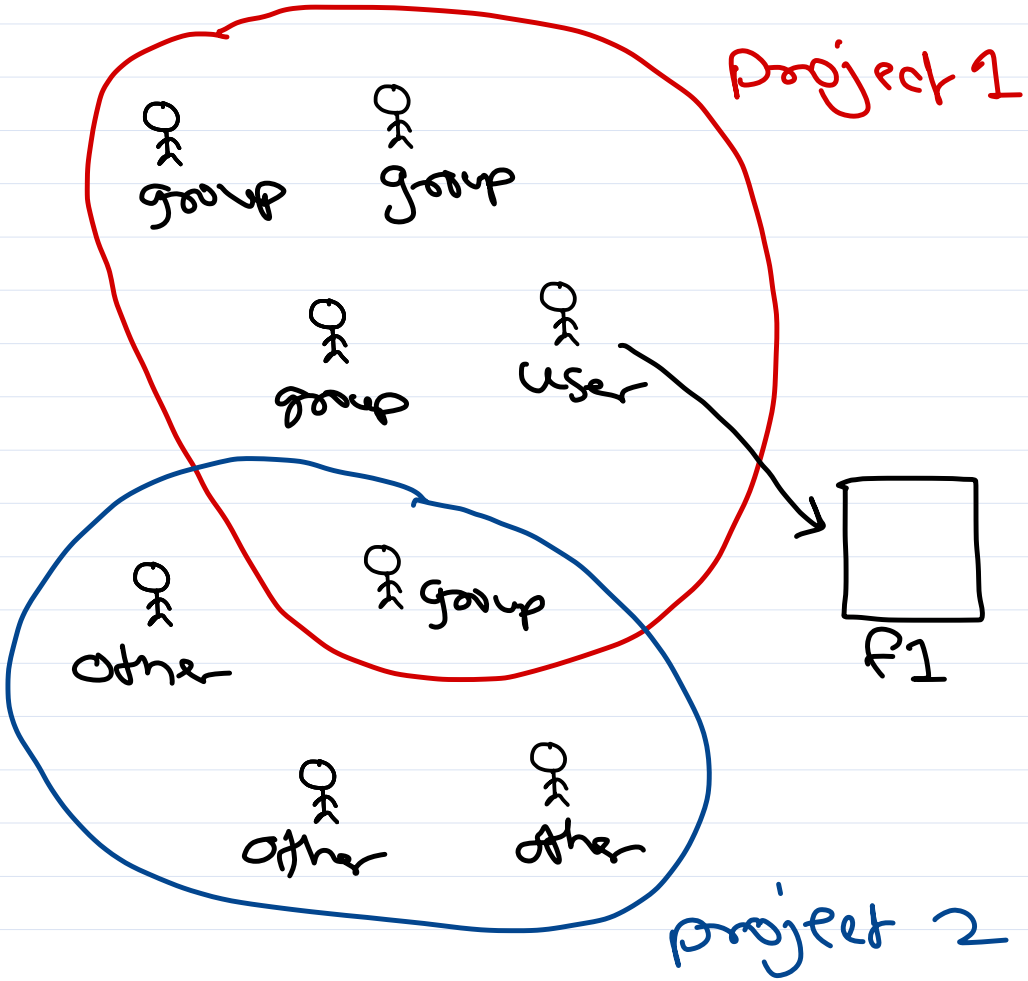
- Computer power on.
- Load firmware (BIOS or EFI) programs from base ROM into RAM.
- Run POST/BIST (from firmware).
- Run Bootstrap loader (from firmware) to find bootable device.
- Bootstrap loader loads bootloader program from bootable device.
- Bootloader program shows multiple options to end user and user select one of them.
- Bootloader program run corresponding bootstrap program.
- Bootstrap program load OS kernel into main memory and OS boot.



Operating System Concepts

Sunbeam Infotech

File permissions



In Linux, users are added into groups. When a new user is created, by default a new group created with same name.

Also there are predefined groups e.g. sudo, root, dial, ...

file → user and group.

file permi at 3 levels:

- user → rwx
- group → rwx
- other → rwx

File permissions

chmod +x filepath ← give perm to ugo
+w
+r

chmod -x filepath ← remove perm from ugo
-w
-r

chmod g+w file ← give/revoke w to group
g-w file

chmod u+w file ← give/revoke w to user
u-w file

chmod o+w file ← give/revoke w to other
o-w file

rwx rwx rwx
u g o

give perm: rwx r-x r--
111 101 100
7 5 4

chmod 754 filepath.

To change user/owner of file:

sudo chown user:group filepath.

(-R) → for all files in dir and subdirs recursively

Booting

① Bootable device
- Storage device whose first sector contains bootstrap pgm.

② Bootstrap pgm
- that loads OS kernel in RAM.

③ bootloader
- resides in 2nd sector of bootable device
- gives options to end user to select OS to boot.
- based on user option start bootstrap program of that OS.

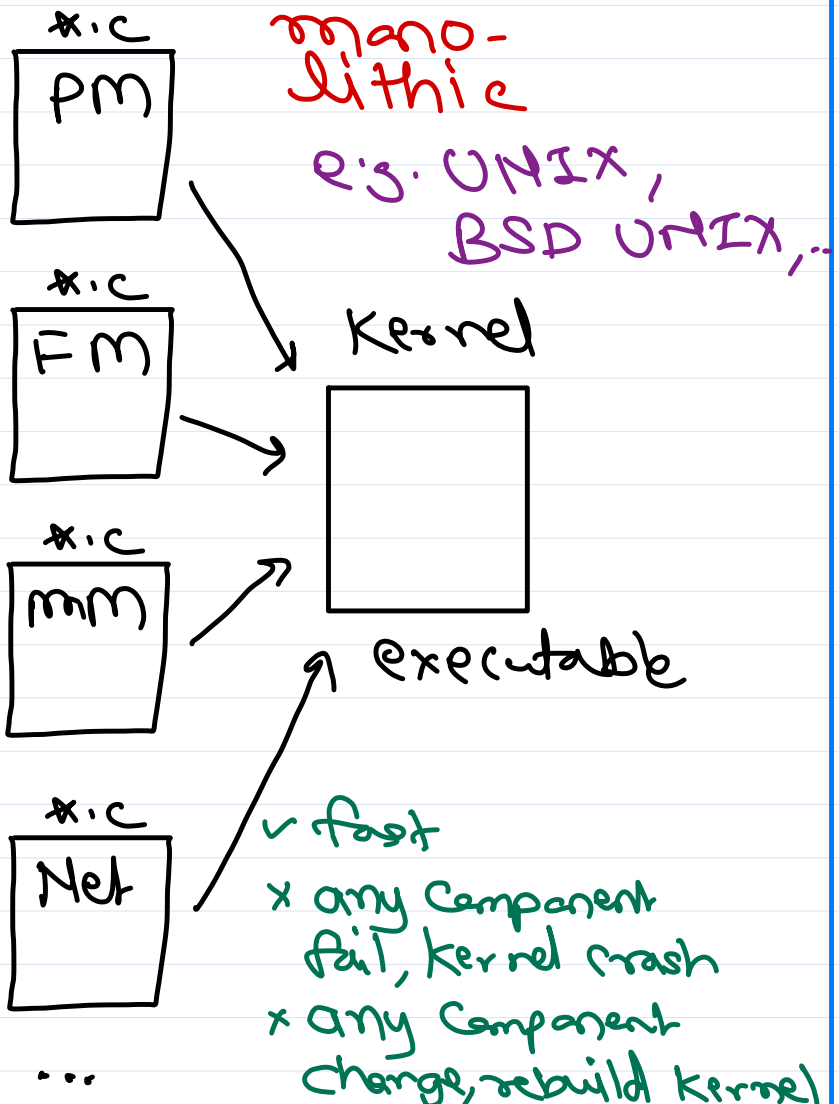
e.g. windows - ntldr (legacy), bootmgr (vista+)

- Linux - (GRUB or LILO)
- Mac OS X - Bootcamp
- Solaris - SILO
- BSD Unix - BTX

④ POST
- Program from BIOS Rom that execute when computer power on.

⑤ Bootstrap loader
- Program from BIOS Rom that find the bootable device i.e. p.d. or disk or ...

OS Kernel Types



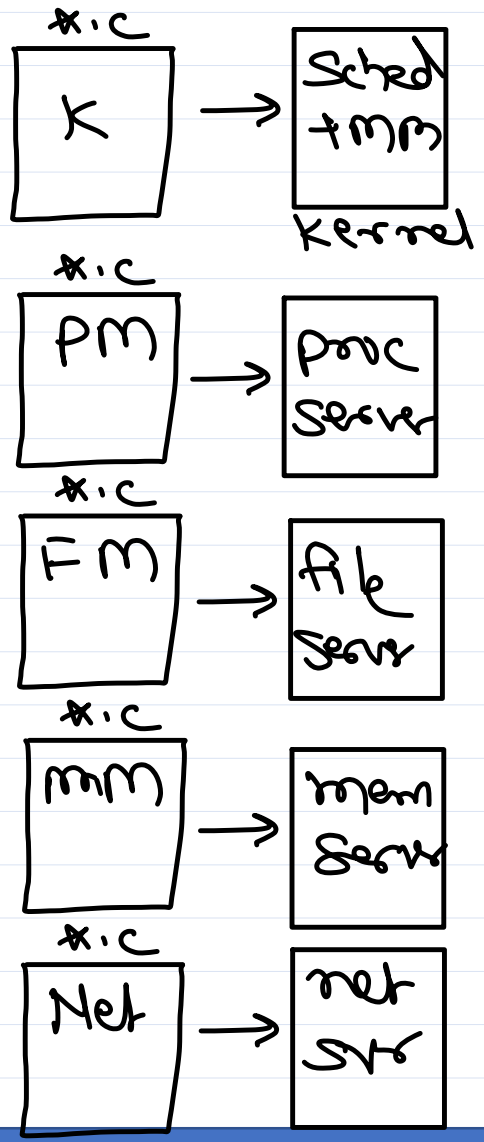
mono-lithic

e.g. UNIX, BSD UNIX,...

✓ fast

✗ any Component fail, kernel crash

✗ any Component change, rebuild kernel



micro Kernel

✓ main kernel is small

✓ other fun... separate exe (servers)

✓ Kernel & Servers Communicate-IPC.

✓ Robust

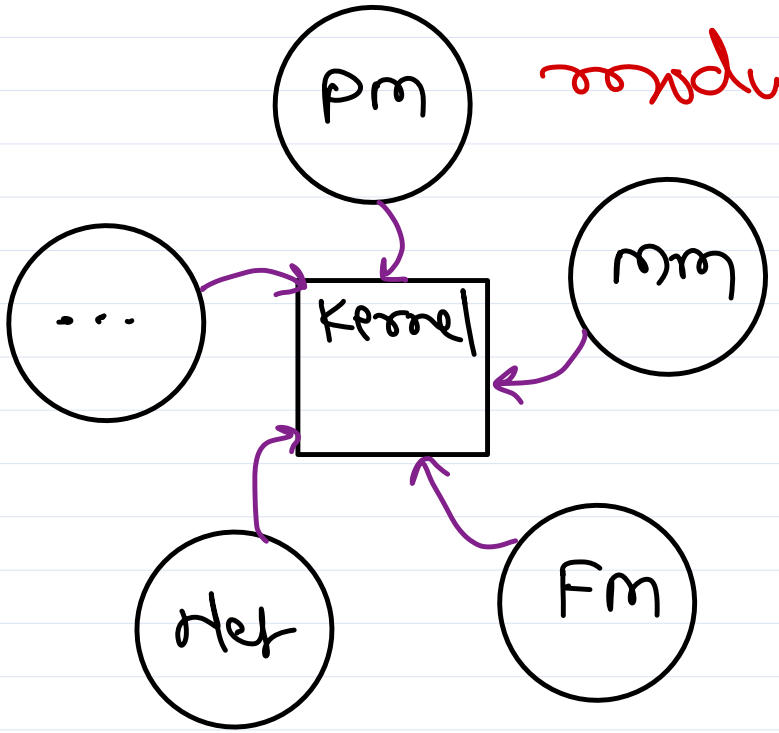
✓ any Component fail, only the Comp stop.

✓ any Component change, rebuild only that Comp.

✗ Slower exec.

e.g. Symbian, MACH, QNX, ...

OS Kernel Types



modular, e.g. Windows, ...

- ✓ each Component is loadable module.
- ✓ when fn needed, module load in kernel at runtime, & execute in kernel process.

- ✓ since only one process, no Ipc need.
- ✓ execution faster.
- ✓ any Comp. change, only Compile that Comp.
- ✗ any Comp. fails, whole kernel crash.

Hybrid

made form multiple kernel source codes.

e.g. Mac OS X

= BSD Unix a.k.a. Darwin (UCB) → Unix + MACH (CMU) → mm

Linux Kernel

- mainly - monolithic
- static components
  - mem mgmt
  - cpu sched
  - process & thread mgmt
  - system calls
  - ...

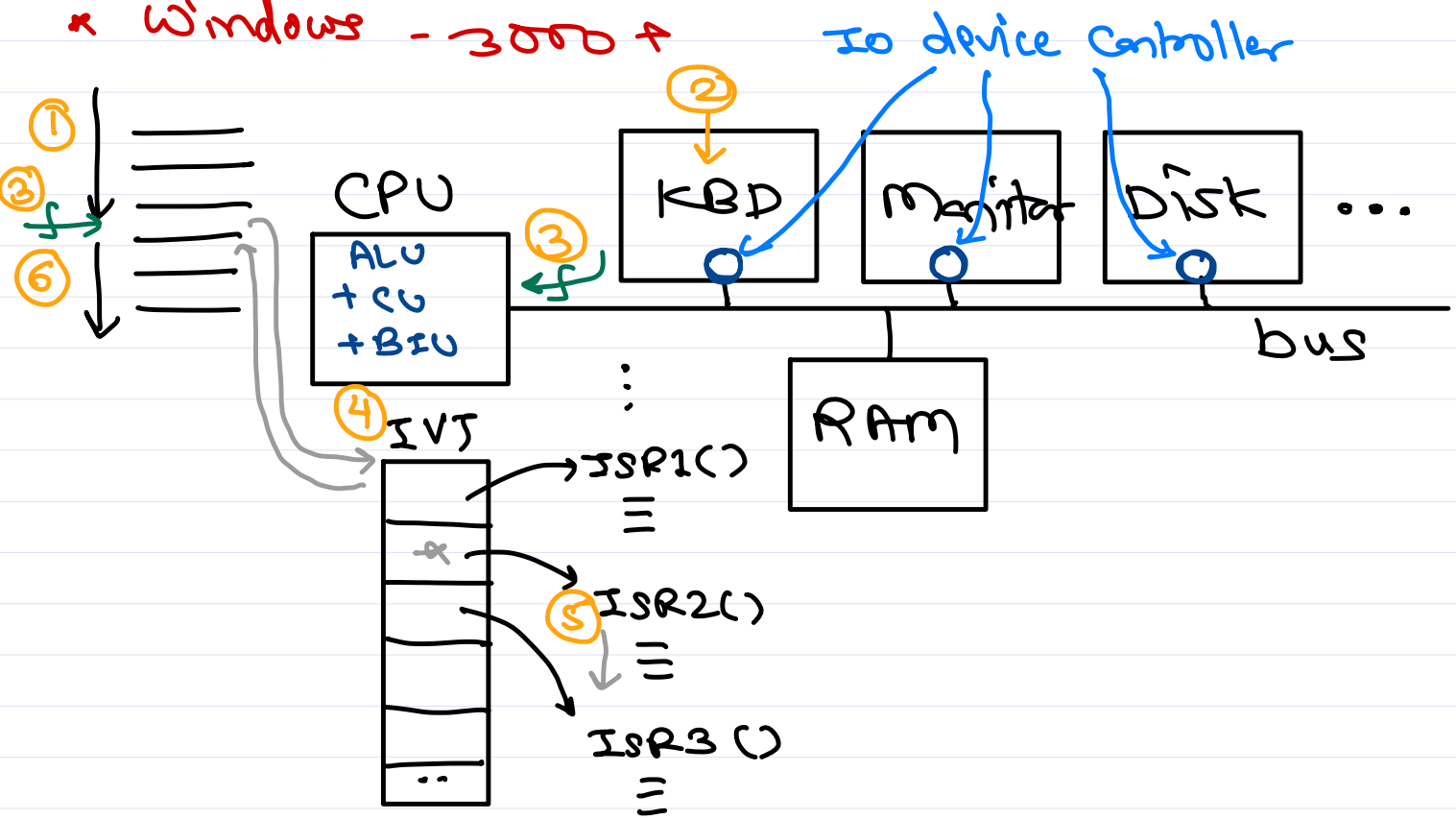
  → vmlinuz (kernel executable) → /boot dir | monolithic
- dynamic components
  - file system (drivers).
  - device drivers.

  → *.ko (kernel modules) → /lib/modules dir | modular.

* kernel source code: www.kernel.org.
  code repo: github.com

System calls

* fns exposed by kernel so that user pgm access. kernel fn.
* UNIX - 64 sys calls
* Linux - 300+
* Windows - 3000+



Thank you!

Nilesh Ghule <nilesh@sunbeaminfo.com>

Operating System Concepts

Agenda

- syscall internals
- process hierarchy
- orphan & zombie state
- thread vs process
- thread creation
- synchronization
- semaphore vs mutex
- deadlock

syscall internals

- Refer yesterday's slides

Process Creation

- System Calls
  - Windows: CreateProcess()
  - UNIX: fork()
  - BSD UNIX: fork(), vfork()
  - Linux: clone(), fork(), vfork()

fork() syscall

- To execute certain task concurrently we can create a new process (using fork() on UNIX).
- fork() creates a new process by duplicating calling process.
- The new process is called as "child process", while calling process is called as "parent process".
- "child" process is exact duplicate of the "parent" process except few points pid, parent pid, etc.

- pid = fork();
  - On success, fork() returns pid of the child to the parent process and 0 to the child process.
  - On failure, fork() returns -1 to the parent.
- Even if child is copy of the parent process, after its creation it is independent of parent and both these processes will be scheduled sepeately by the scheduler.
- Based on CPU time given for each process, both processes will execute concurrently.

getpid() vs getppid()

- pid1 = getpid(); // returns pid of the current process
- pid2 = getppid(); // returns pid of the parent of the current process

When fork() will fail?

- When no new PCB can be allocated, then fork() will fail.
- Linux has max process limit for the system and the user. When try to create more processes, fork() fails.
- terminal> cat /proc/sys/kernel/pid_max

Orphan process

- If parent of any process is terminated, that child process is known as orphan process.
- The ownership of such orphan process will be taken by "init" process.

Zombie process

- If process is terminated before its parent process and parent process is not reading its exit status, then even if process's memory/resources is released, its PCB will be maintained. This state is known as "zombie state".
- To avoid zombie state parent process should read exit status of the child process. It can be done using wait() syscall.

wait() syscall

- ret = wait(&s);
  - arg1: out param to get exit code of the process.
  - returns: pid of the child process whose exit code is collected.

- wait() performs 3 steps:
  - Pause execution parent until child process is terminated.
  - Read exit code from PCB of child process & return to parent process (via out param).
  - Release PCB of the child process.
- The exit status returned by the wait() contains exit status, reason of termination and other details.
- Few macros are provided to access details from the exit code.
  - WEXITSTATUS()

waitpid() syscall

- This extended version of wait() in Linux.
- ret = waitpid(child_pid, &s, flags);
  - arg1: pid of the child for which parent should wait.
    - -1 means any child.
  - arg2: out param to get exit code of the process.
  - arg3: extra flags to define behaviour of waitpid().
- returns: pid of the child process whose exit code is collected.
  - -1: if error occurred.

exec() syscall

- exec() syscall "loads a new program" in the calling process's memory (address space) and replaces the older (calling) one.
- If exec() succeed, it does not return (rather new program is executed).
- There are multiple functions in the family of exec():
  - execl(), execlp(), execle(),
  - execv(), execvp(), execve(), execvpe()
- exec() family multiple functions have different syntaxes but same functionality.

Thread concept

- Threads are used to execute multiple tasks concurrently in the same program/process.
- Thread is a light-weight process.
  - For each thread new control block and stack is created. Other sections (text, data, heap, ...) are shared with the parent process.

    - Inter-thread communication is much faster than inter-process communication.
    - Context switch between two threads in the same process is faster.
- Thread stack is used to create function activation records of the functions called/executed by the thread.

Process vs Thread

- In modern OS, process is a container holding resources required for execution, while thread is unit of execution/scheduling.
- Process holds resources like memory, open files, IPC (e.g. signal table, shared memory, pipe, etc.).
- PCB contains resources information like pid, exit status, open files, signals/ipc, memory info, etc.
- CPU time is allocated to the threads. Thread is unit of execution.
- TCB contains execution information like tid, scheduling info (priority, sched algo, time left, ...), Execution context, Kernel stack, etc.
- terminal> ps -e -o pid,nlwp,cmd
- terminal> ps -e -m -o pid,tid,nlwp

main thread

- For each process one thread is created by default called as main thread.
- The main thread executes entry-point function of the process.
- The main thread use the process stack.
- When main thread is terminated, the process is terminated.
- When a process is terminated, all threads in the process are terminated.

thread functions

- pthread_create()
    - Create a new thread.
        - arg1: posix thread id (out param)
        - arg2: thread attributes -- NULL means default attributes
            - stack size
            - scheduling policy
            - priority
        - arg3: address of thread function

        - void* thread_function(void*);
            - arg1: void* -- param to the thread function (can be of any type, array or struct).
            - returns: void* -- result of thread (can be of any type)
        - arg4: param to thread function
        - returns: 0 on success.

- Join thread
    - The current thread wait for completion of given thread and will collect return value of that thread.
    - pthread_join(th_id, &res);
        - arg1: given thread (for which current thread is to blocked).
        - arg2: address of result variable (out param to collect result of the given thread)

- Thread termination
    - When thread function is completed, the thread is terminated.
    - To terminate current thread early use pthread_exit() function.
    - pthread_exit(result);
        - arg1: result (void*) of the current thread

- Thread cancellation
    - A thread in a process can request to cancel execution of another thread.
    - pthread_cancel(tid)
        - arg1: id of the thread to be cancelled.

Threading models

- Threads created by thread libraries are used to execute functions in user program. They are called as "user threads".
- Threads created by the syscalls (or internally into the kernel) are scheduled by kernel scheduler. They are called as "kernel threads".
- User threads are dependent on the kernel threads. Their dependency/relation (managed by thread library) is called as "threading model".
- There are four threading models:
    - Many to One
    - Many to Many

    - One To One
    - Two Level Model
- Many to One
    - Many user threads depends on single kernel thread.
    - If one of the user thread is blocked, remaining user threads cannot function.
- Many to Many
    - Many user threads depend on equal or less number of kernel threads.
    - If one of the user thread is blocked, other user thread keep executing (based on remaining kernel threads).
- One To One
    - One user thread depends on one kernel thread.
- Two Level Model
    - OS/Thread library supports both one to one and many to many model

Synchronization

- Multiple processes accessing same resource at the same time, is known as "race condition".
- When race condition occurs, resource may get corrupted (unexpected results).
- Peterson's problem, if two processes are trying to modify same variable at the same time, it can produce unexpected results.
- Code block to be executed by only one process at a time is referred as Critical section. If multiple processes execute the same code concurrently it may produce undesired results.
- To resolve race condition problem, one process can access resource at a time. This can be done using sync objects/primitives given by OS.
- OS Synchronization objects are:
    - Semaphore, Mutex

Semaphore

- Semaphore is a sync primitive given by OS.
- Internally semaphore is a counter. On semaphore two operations are supported:
    - wait operation: dec op: P operation:
        - semaphore count is decremented by 1.
        - if cnt < 0, then calling process is blocked.
        - typically wait operation is performed before accessing the resource.

    - signal operation: inc op: V operation:
        - semaphore count is incremented by 1.
        - if one or more processes are blocked on the semaphore, then one of the process will be resumed.
        - typically signal operation is performed after releasing the resource.
- Semaphore types
    - Counting Semaphore
        - Allow "n" number of processes to access resource at a time.
        - Or allow "n" resources to be allocated to the process.
    - Binary Semaphore
        - Allows only 1 process to access resource at a time or used as a flag/condition.

Mutex

- Mutex is used to ensure that only one process can access the resource at a time.
- Functionally it is same as "binary semaphore".
- Mutex can be unlocked by the same process/thread, which had locked it.

Semaphore vs Mutex

- S: Semaphore can be decremented by one process and incremented by same or another process.
- M: The process locking the mutex is owner of it. Only owner can unlock that mutex.
- S: Semaphore can be counting or binary.
- M: Mutex is like binary semaphore. Only two states: locked and unlocked.
- S: Semaphore can be used for counting, mututal exclusion or as a flag.
- M: Mutex can be used only for mutual exclusion.

Deadlock

- Deadlock occurs when four conditions/characteristics hold true at the same time.
    - No preemption: A resource should not be released until task is completed.
    - Mutual exclusion: Resources is not sharable.
    - Hold & Wait: Process holds a resource and wait for another resource.
    - Circular wait: Process P1 holds a resource needed for P2, P2 holds a resource needed for P3 and P3 holds a resource needed for P1.

Deadlock Prevention

- OS syscalls are designed so that at least one deadlock condition does not hold true.
- In UNIX multiple semaphore operations can be done at the same time.

Deadlock Avoidance

- Processes declare the required resources in advanced, based on which OS decides whether resource should be given to the process or not.
- Algorithms used for this are:
  - Resource allocation graph: OS maintains graph of resources and processes. A cycle in graph indicate circular wait will occur. In this case OS can deny a resource to a process.
  - Banker's algorithm: A bank always manage its cash so that they can satisfy all customers.
  - Safe state algorithm: OS maintains statistics of number of resources and number processes. Based on stats it decides whether giving resource to a process is safe or not (using a formula):
    - Max num of resources required < Num of resources + Num of processes
      - If condition is true, deadlock will never occur.
      - If condition is false, deadlock may occur

IPC overview

- A process cannot access of memory of another process directly. OS provides IPC mechanisms so that processes can communicate with each other.
- IPC models
  - Shared memory model
    - Processes write/read from the memory region accessible to both the processes.
    - OS only provides access to the shared memory region.
  - Message passing model
    - Process send message to the OS and the other process receives message from the OS.
    - This is slower compared to shared memory model.
- Unix/Linux IPC mechanisms
  - Signals
  - Shared memory
  - Message queue

    - Pipe
    - Socket

VI Editor

- Refer slides
- sudo apt-get install vim
- VI editor works in two modes
    - command mode
    - insert mode
- press i - to go into insert mode
- press Esc - to go into command node
- VI editor commands:
    - :w - write/save into file
    - :q - quit vi editor
    - :wq - save and quit
    - :y - to copy current line
    - yy - to copy current line
    - nyy - copy n lines from current line
    - :m,ny - copy fomr mth line to nth line
    - :d - to cut current line
    - dd - to cut current line
    - ndd - cut n lines from current line
    - :m,nd - cut fomr mth line to nth line

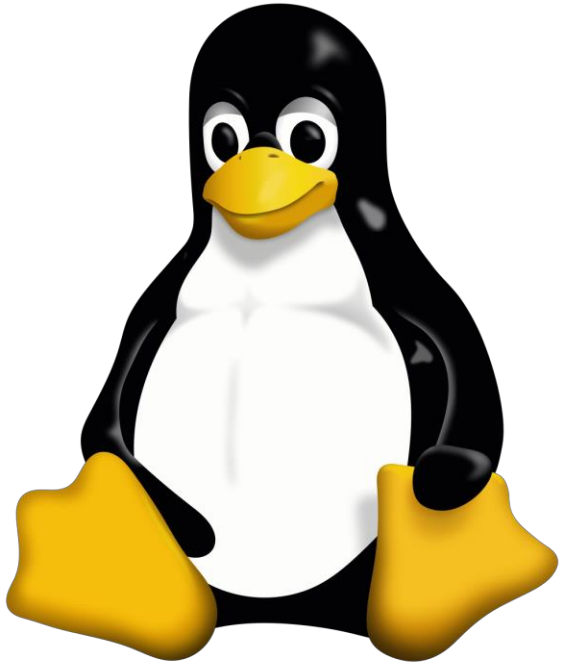
    - press p - to paste copied line on next line of current line

- To do setting in vi editor
    - create file into home directory as below:

```
vim ~/.vimrc
```

- add below content into it

```
set number
set autoindent
set tabstop=4
set nowrap
```

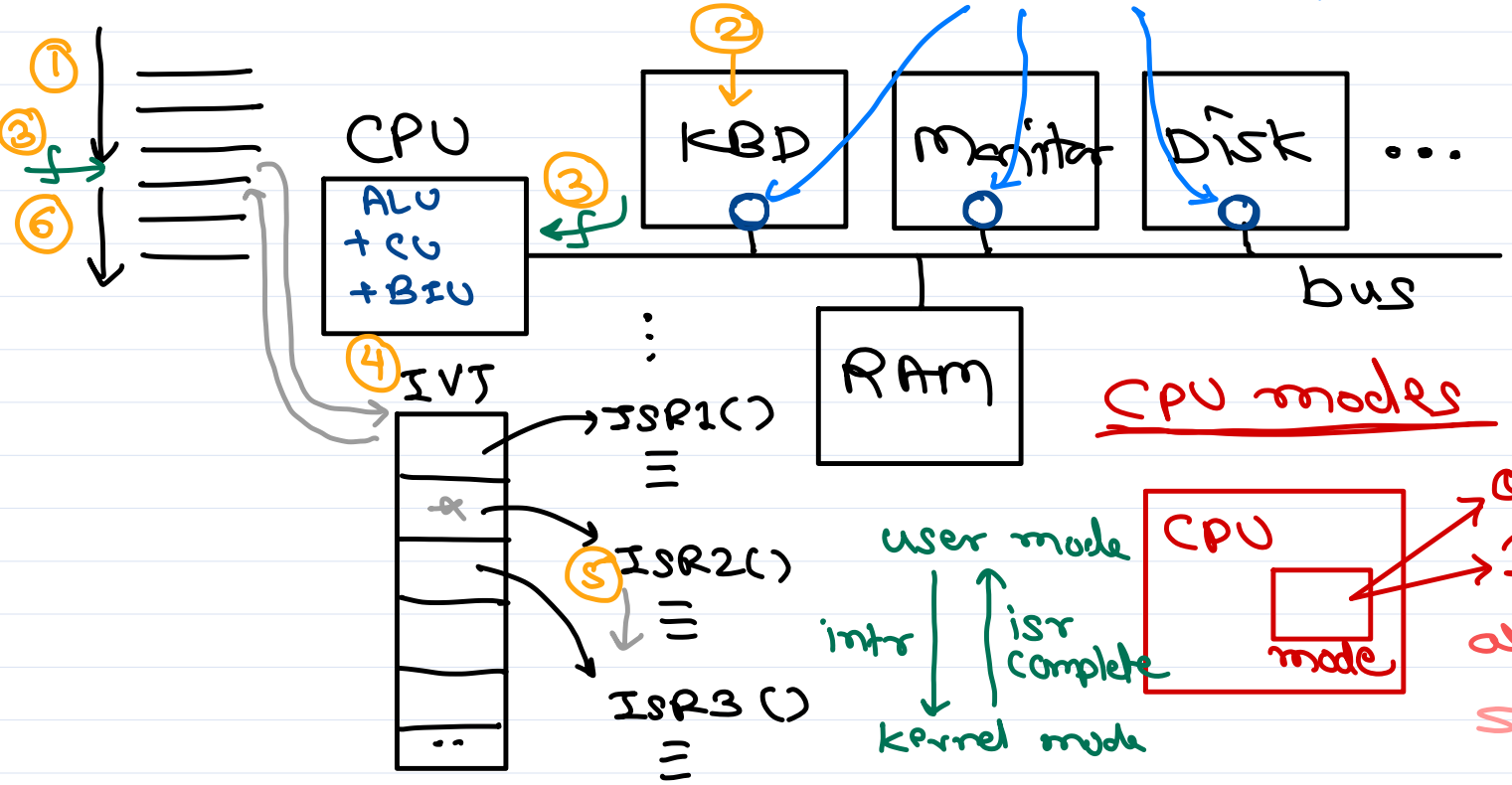


Operating System Concepts

Sunbeam Infotech

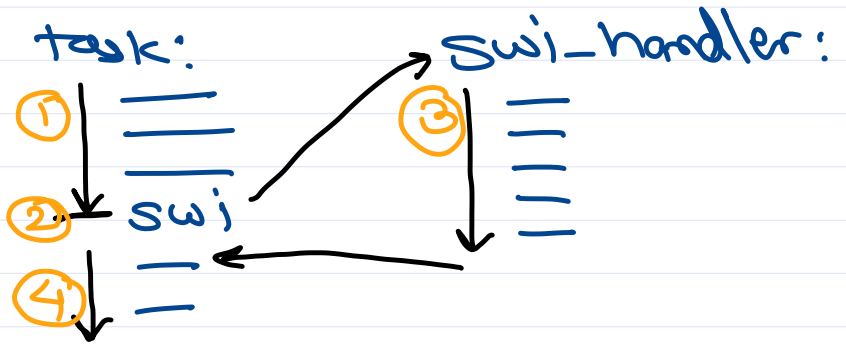
Interrupts and CPU modes

* fns exposed by kernel so that user pgm access. kernel fn.
* UNIX - 64 sys calls
* Linux - 300+
* Windows - 3000+



Interrupts
- Sent by IO devices
- hw intr
- sw intr ← special instruction (CPU specific)

e.g. ARM → SWI or SVC
x86 → INT

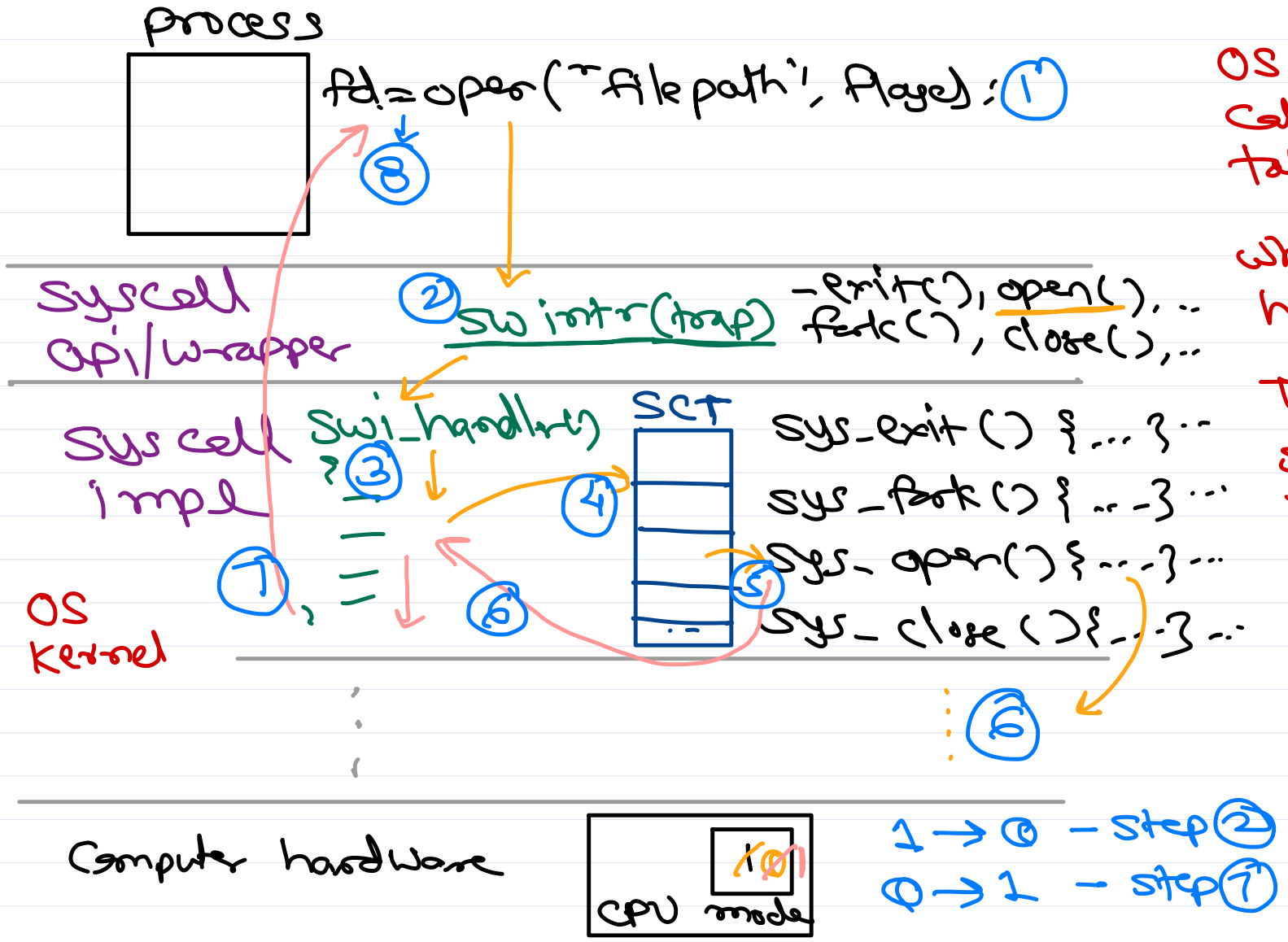


CPU modes

0 : kernel / system / monitor / supervisor privileged mode.
1 : user / unprivileged mode.

all regrs accessible & all ops allowed
special instns not allowed e.g. intr handling, in/out, ...

System calls



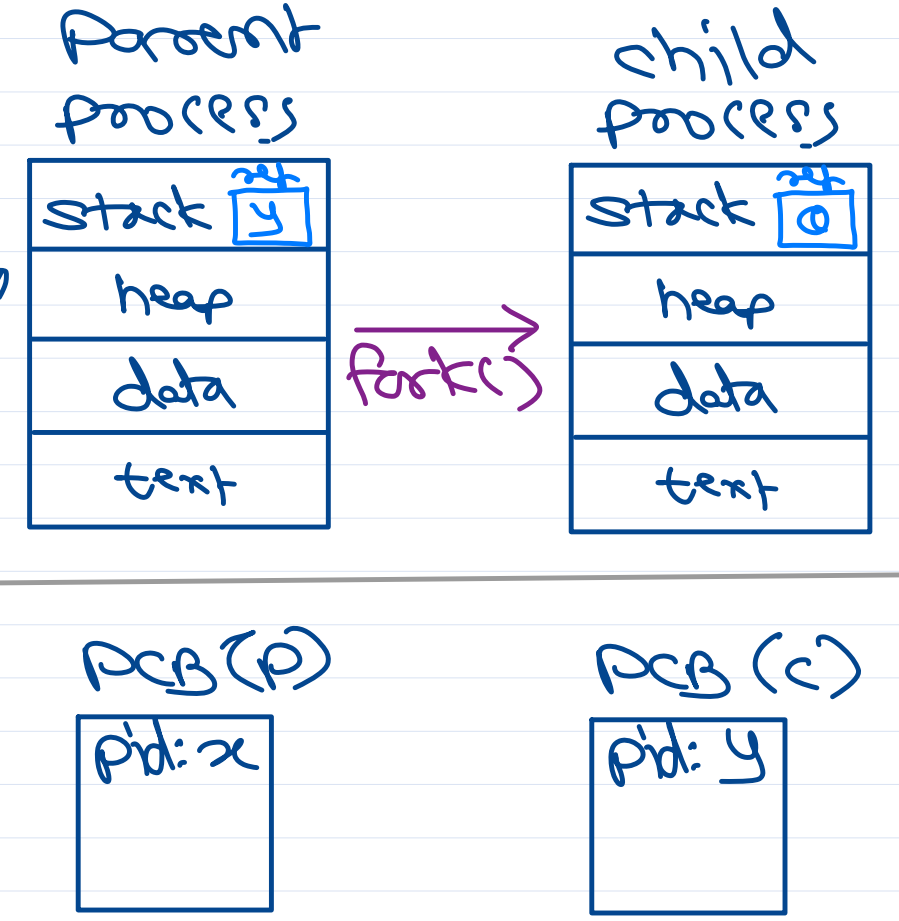
OS stores addresses of all sys call impl into a syscall table (SCT).

When sw intr occurs, swi handler is executed (by hw).

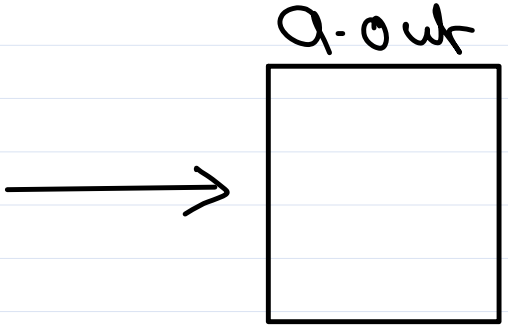
This handler gets addr of syscall from SCT and invoke it.

fork() syscall

* fork() creates a new process by duplicating calling process. The new process is called child; while calling process is called parent.
* child & parent runs in separate memory space
* child & parent are almost same except...
  - a) child pid is different than parent pid.
  - b) ...
* fork() returns 0 in child process; while it returns pid of child in parent process. If failed, fork() returns -1 in parent.

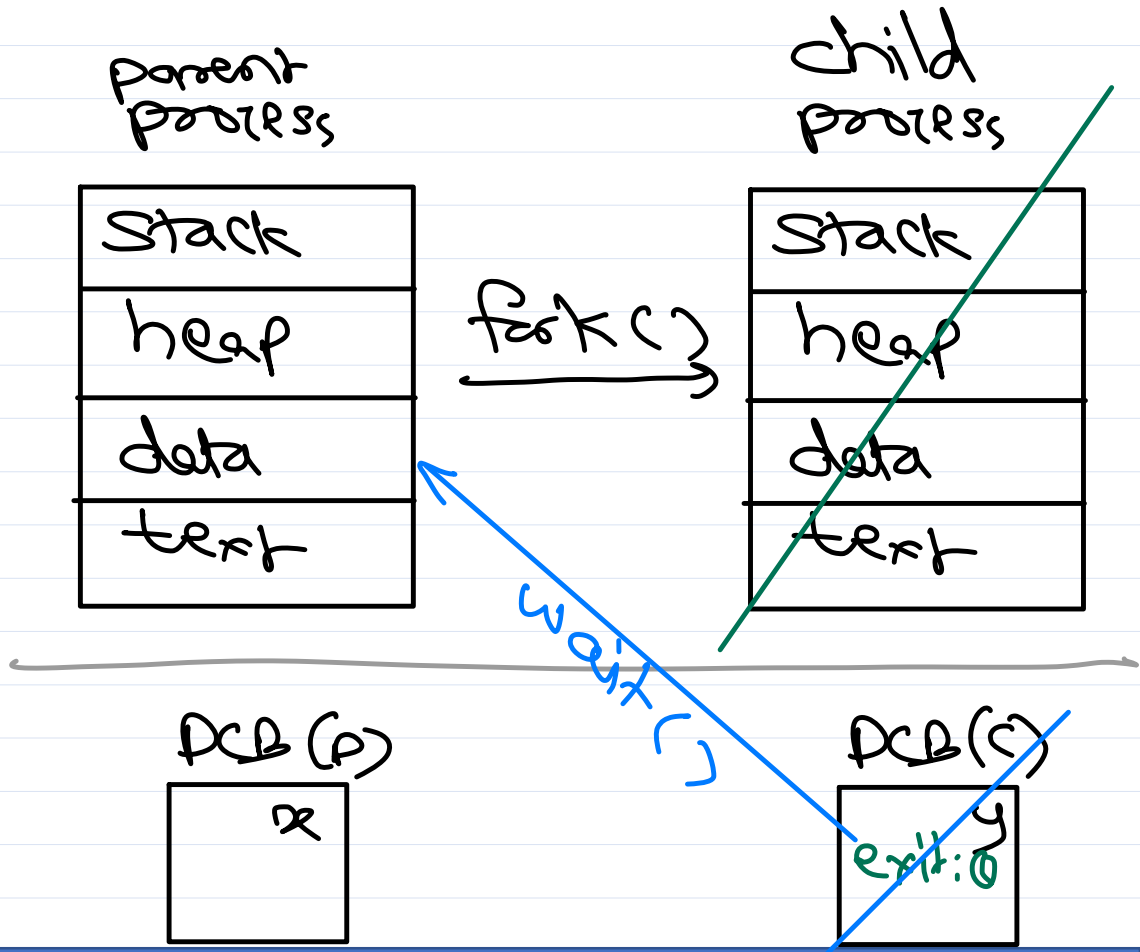



```
int main() {
    int ret;
    —
    ret = fork();
    =
}
```



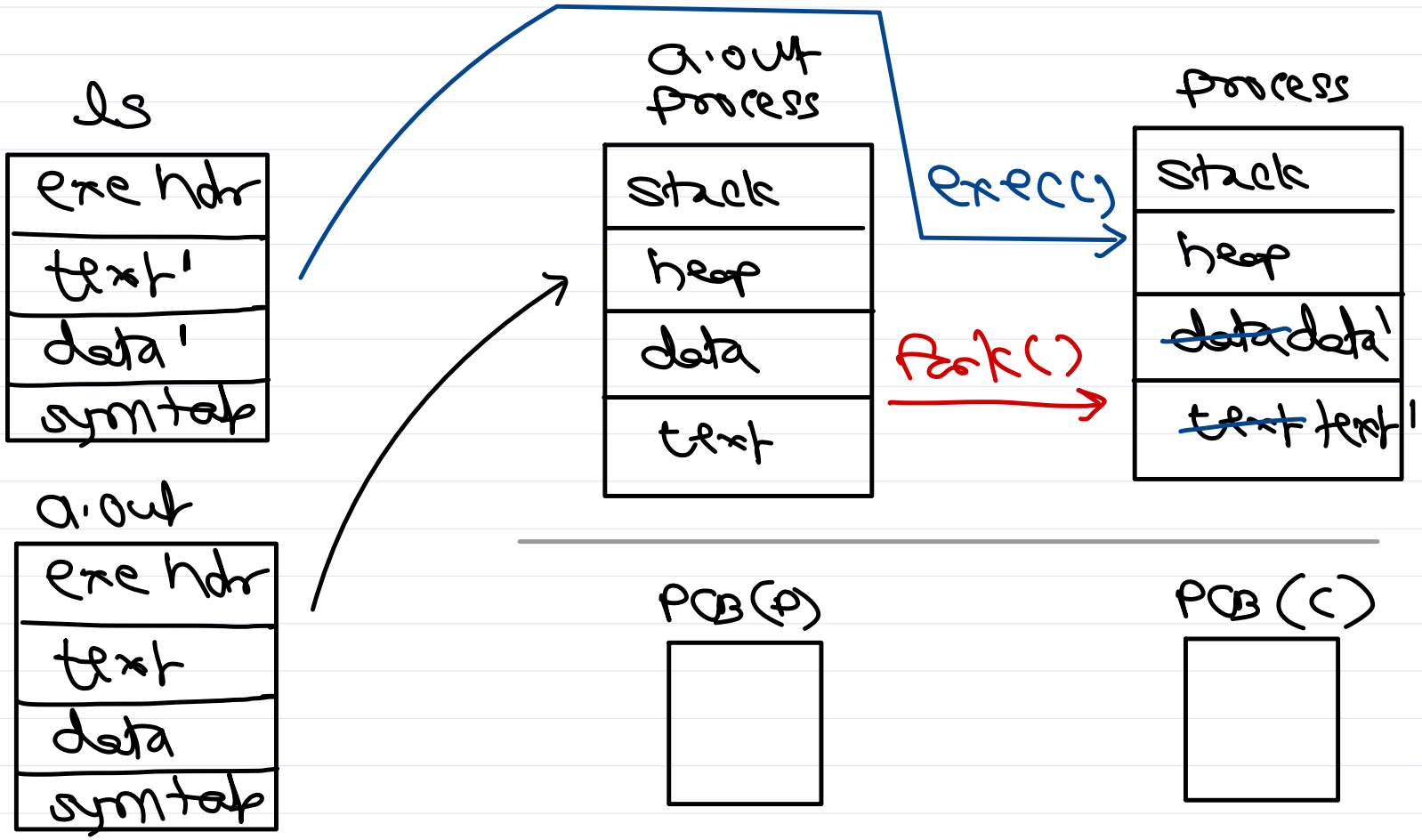
Orphan and Zombie process

- when parent is terminated before child process, the child is said to be orphan.
- ownership/parentship of such child process is handed over to init/systemd (pid=1).
- when child is terminated before parent and parent is not reading exit status of the child, the child process is in zombie state
- In this state all resources of child are released except the PCB.
- The parent must read exit status of child from that PCB, to make process dead(x).
- This is done using wait() sys call,
  1. pause execution of parent until one of its child is terminated
  2. read exit status of child from its PCB.
  3. release PCB of child



exec() syscall

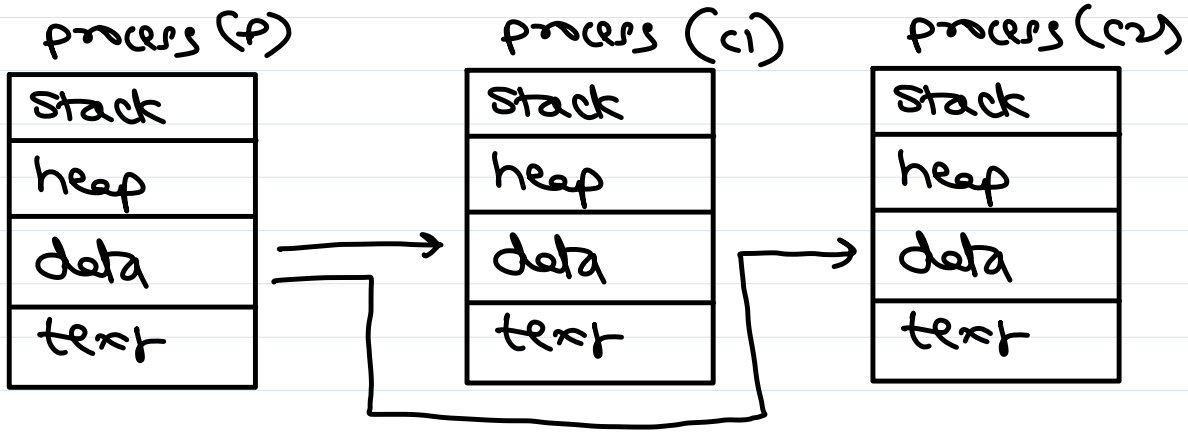
exec() loads a new program in calling process's memory replacing old program image.

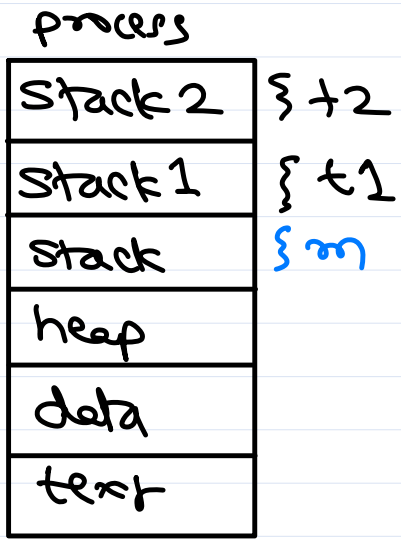



```
r = fork();
if(r==0) {
    e = execl("/bin/ls",
        "ls", "-l", 0);
} else {
    wait(&s);
}
```

Multi-threading

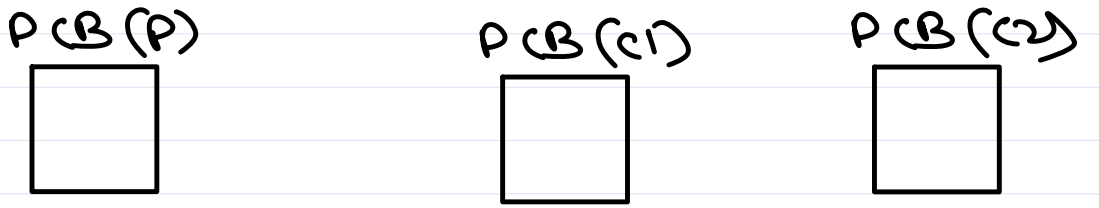
* Threads are used to do multiple tasks concurrently within a process.
* Thread is a light-weight process.

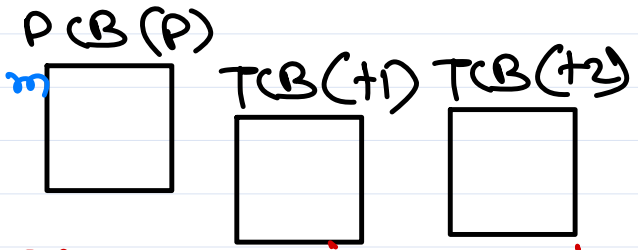




process is a container that holds resources required for execution; while threads are unit of exec/ scheduling.

* PCB contains resource info:
  - pid, exit, mem info, ipc info, files info, ...
* TCB contains exec. info:
  - tid, sched info, state, kernel stack, exec ctx.





for each process one thread created by default called as main thread.

for each thread a new stack & a new TCB is created. The remaining sections are shared with parent process.

Thread

POSIX thread lib.

step1 → create a thread fn.

```
void* thread_func(void *param) {
}
```

step2 → create a thread

```
ret = pthread_create(&th, NULL, thread_func, NULL);
```

arg1: thread id (out param)

arg2: thread attrib (stack size, priority, sched, ...)

NULL = default attrib

arg3: func to be executed by thread.

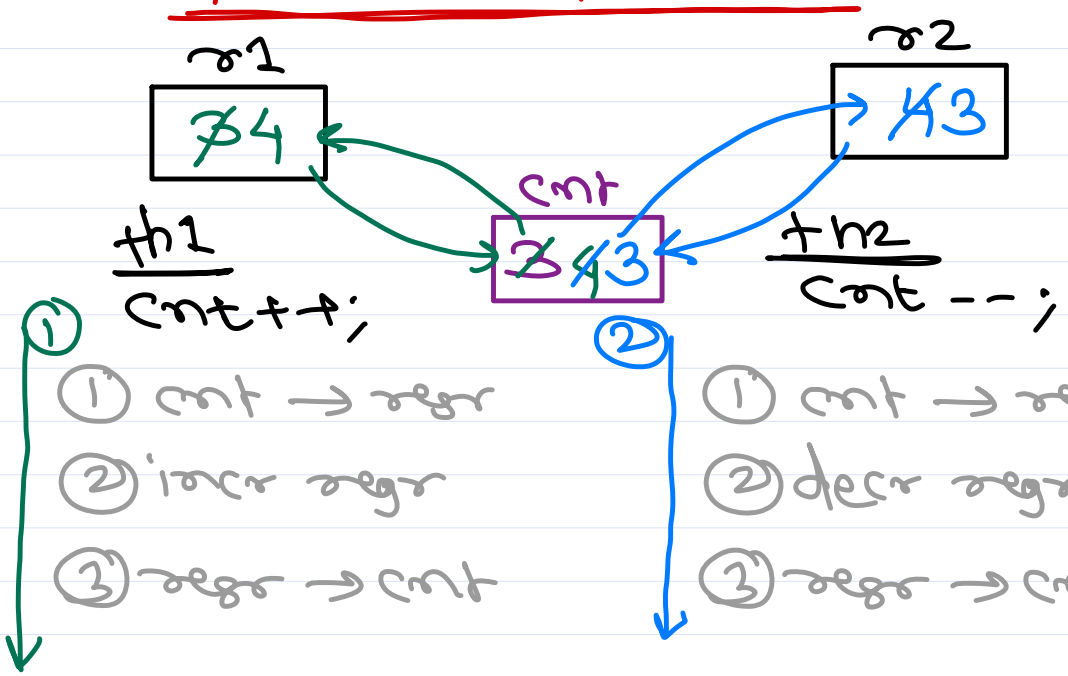
arg4: arg to thread func.

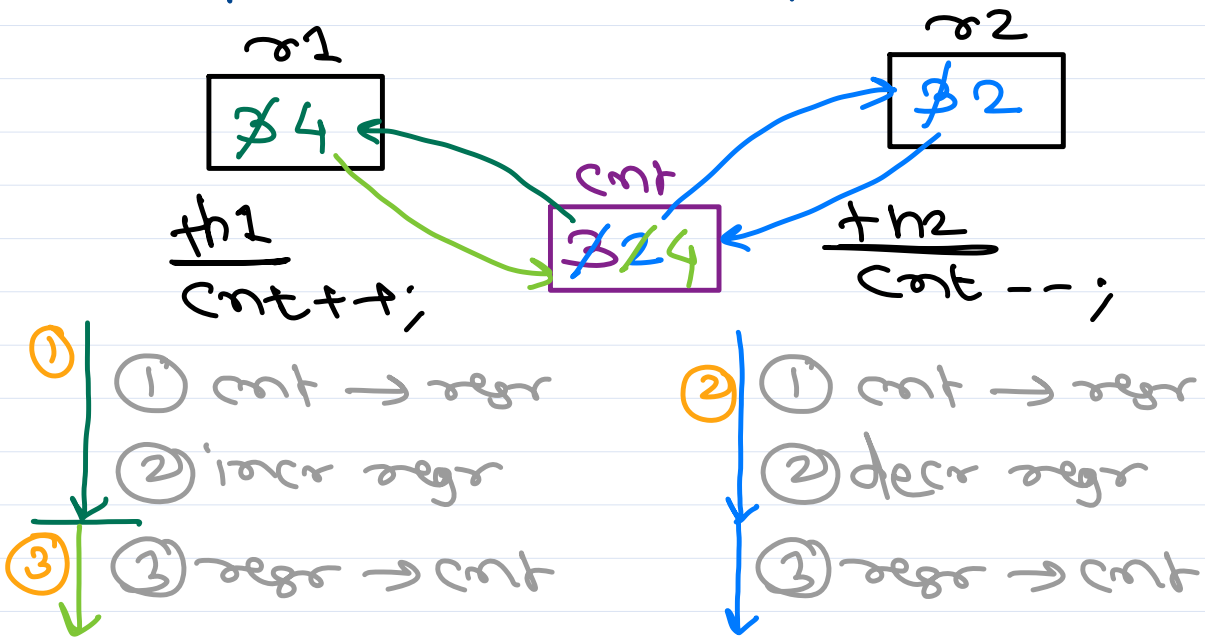
*POSIX - Unix Std (by IEEE)

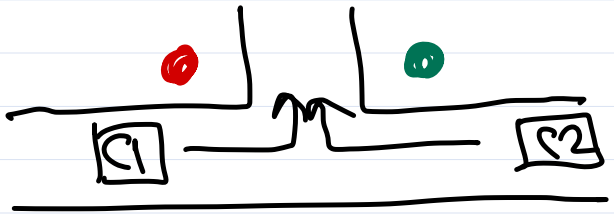
↳ Portable Operating System Interface for Xwindows

Race condition and Synchronization

When multiple threads try to access same resource at the same time, then race condn occurs. It may lead to unexpected results.



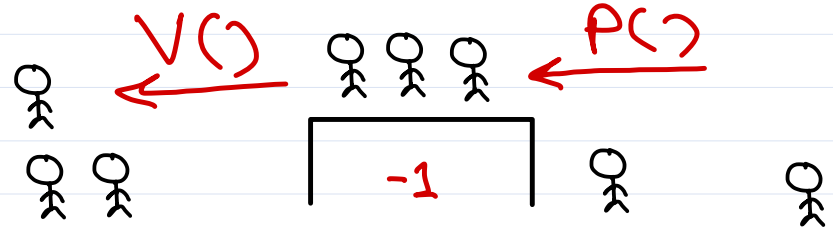




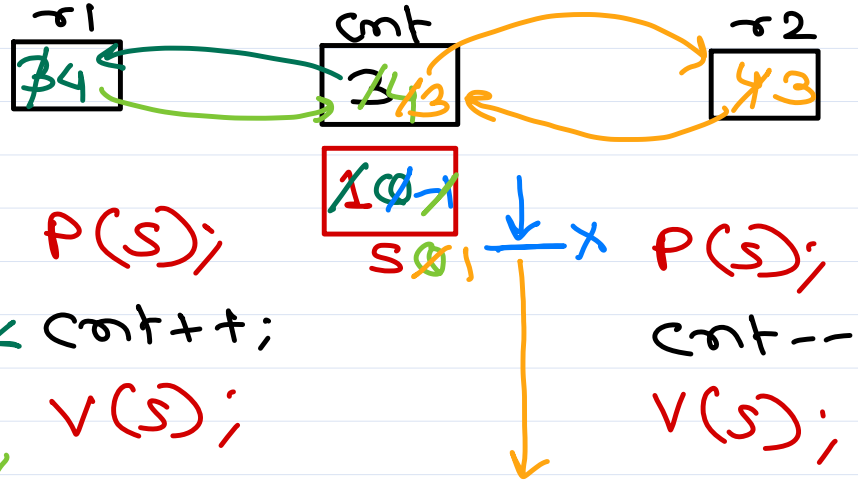
To solve race cond, only one process/thread should be given access to resource while blocking others. this is called as "synchronization".

Semaphore vs Mutex

- Dijkstra
- Semaphore is counter.
- dec op - wait op - P op
  - decr cnt by 1.
  - if cnt < 0, block cur process/thread.
- inc op - signal op - V op
  - incr cnt by 1.
  - if one/more threads/processes are blocked, then wake up one of them.



- Semaphore types
  - counting sema
  - binary sema



Sema applns:

1. Counting - resource/ processes
2. mutual exclusion
3. Condition / event

Mutex: specialized sync obj for mutual exclusion. Two states: locked / unlocked.
Operations: lock(m); unlock(m);

th1
```
lock(m);
== Res
unlock(m);
```

th2
```
lock(m);
== Res
unlock(m);
```

Thread locking the mutex become owner of the mutex and only owner can unlock it.

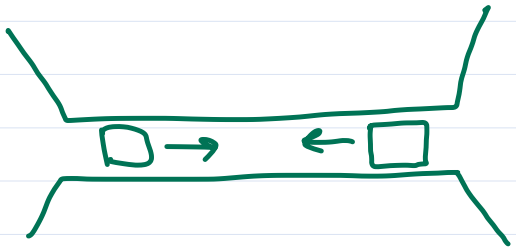
Deadlock

program 1

1. open file
2. connect printer
3. print file
4. disconnect prn
5. close file

program 2

1. connect printer
2. open file
3. print file
4. close file
5. disconnect prn

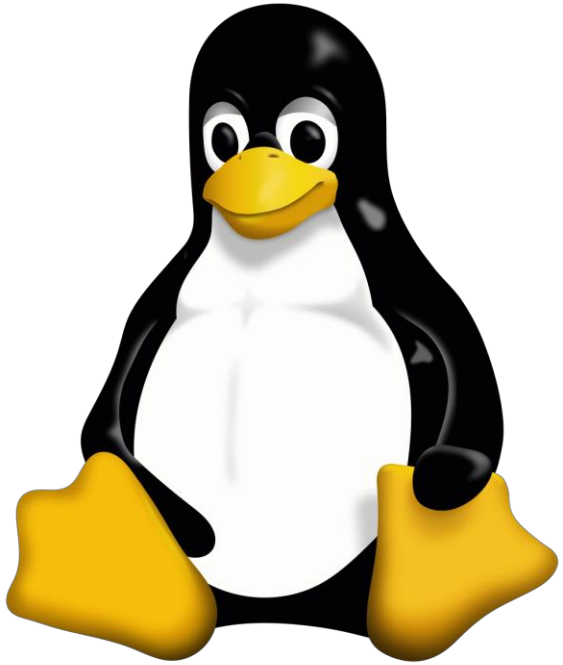


deadlock characteristics

1. no preemption
2. mutual exclusion
3. hold & wait
4. circular wait

Thank you!

Nilesh Ghule <nilesh@sunbeaminfo.com>

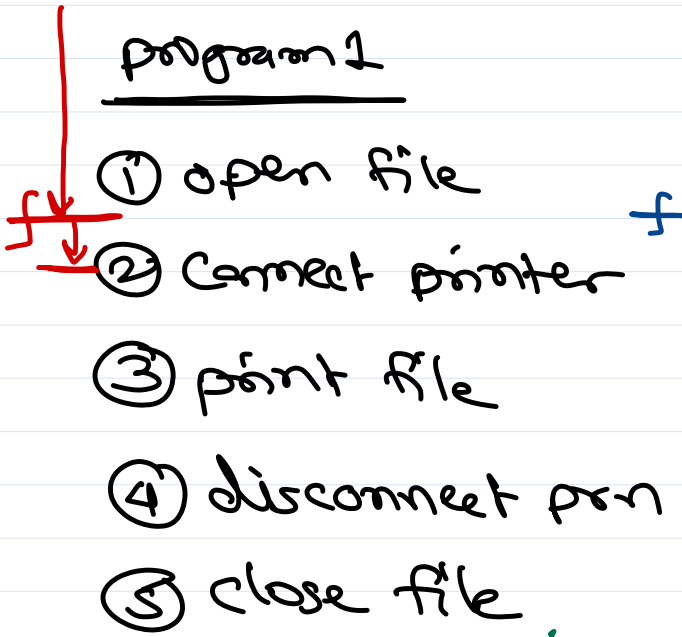


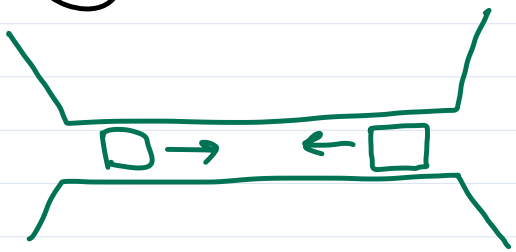
Operating System Concepts

Sunbeam Infotech

Deadlock

deadlock: processes involved in deadlock are permanently waiting
Starvation: process in ready queue not get cpu time due to low pri. (in wait que).





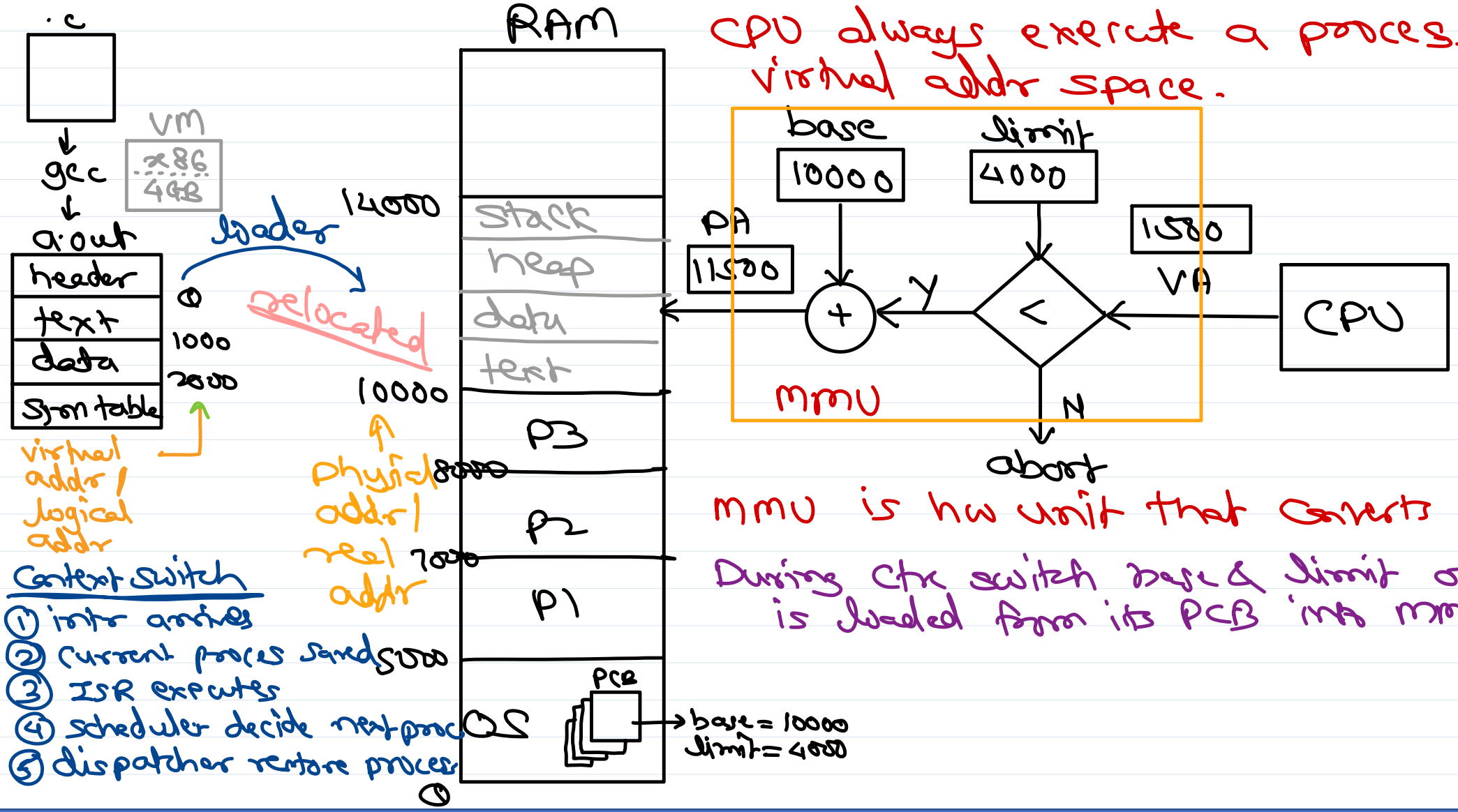
deadlock recovery:
forcible preempt resources & then reassign to processes.

deadlock characteristics
1. no preemption
2. mutual exclusion
3. hold & wait
4. circular wait

deadlock prevention:
- design system (OS + app) in a way that atleast one deadlock cond never holds true.
UNIX OS provide this facility-
can operate multiple sem at once.
i.e. acquire multiple resources at once ⇒ No Hold & wait.

deadlock avoidence:
- processes first informs OS about resources that it will need.
- later processes req resources to OS as and when needed.
- when process req resource, OS checks if this may result in deadlock and if there are chances, OS may deny the req upto some timeout.
- algorithms:
  1. Safe state 2. resource alloc graph
  3. banker's algo.

Memory management



Memory management schemes

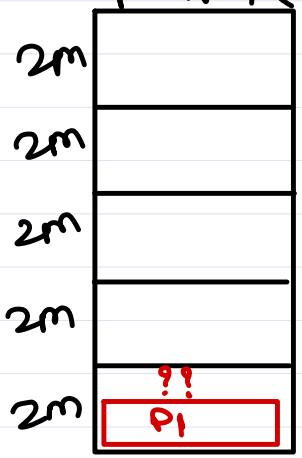
MM schemes depends on avail MMU hw:

MMU hw	MM scheme
① simple	① Contiguous alloc
② segmentation	② segmentation
③ paging	③ paging

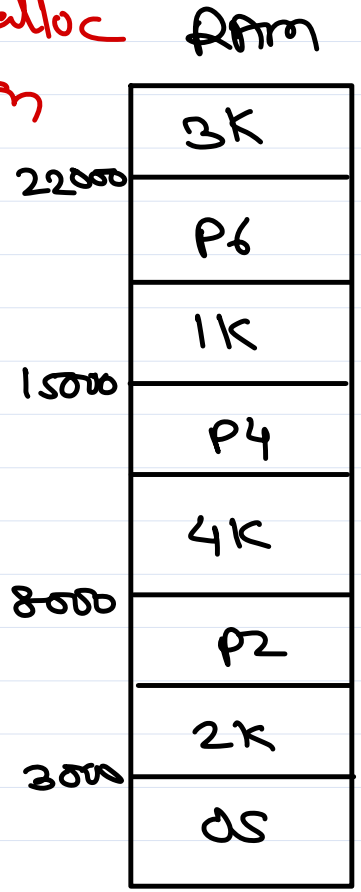
Contiguous allocation:

① fixed Part. method.

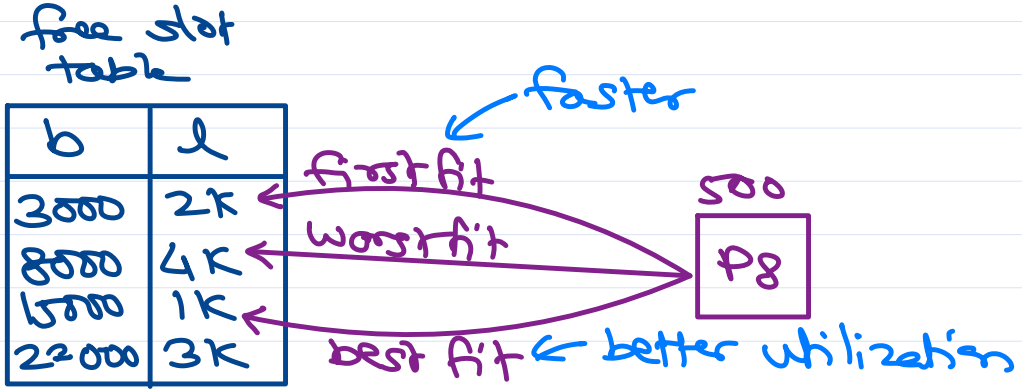
RAM (10M) - fixed num of fixed sized parts.



- 1 proc → 1 Part.
- ✓ simple impl.
- ✗ max processes = num of parts.
- ✗ max proc size = max part size.
- ✗ internal frag.

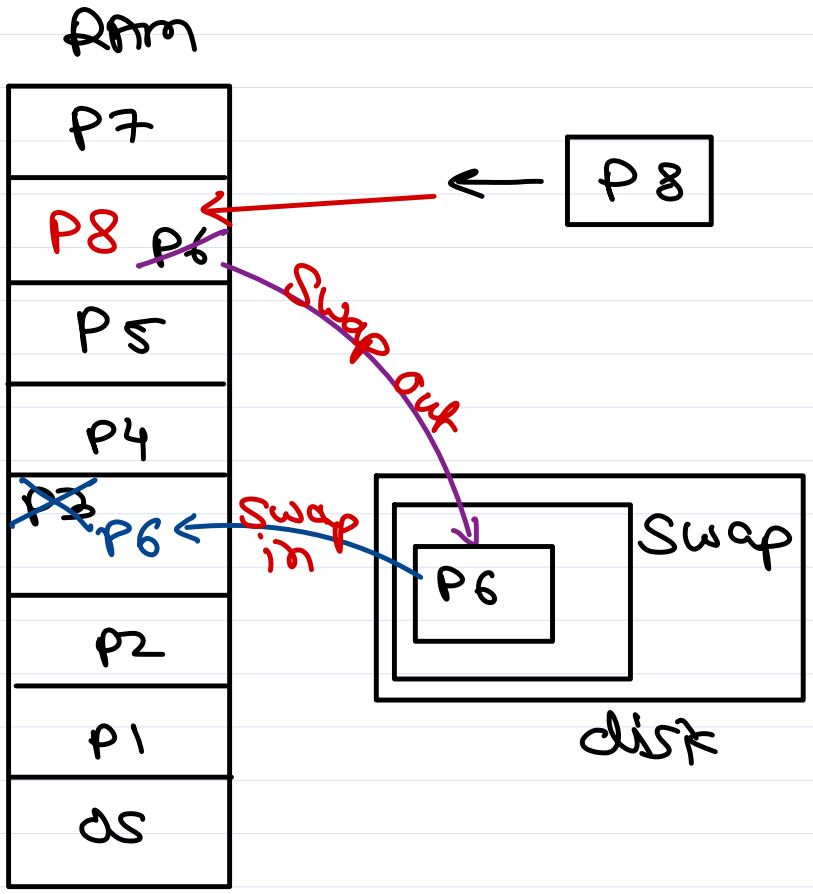


② variable part method/ dynamic alloc method



- ✓ process → avail slot
- ✓ simple (w.r.t. seg. & paging)
- ✗ num of processes = avail RAM size.
- ✗ max proc size = avail RAM size.
- ✓ no 'internal frag.
- ✗ external frag → soln: compaction.

Virtual memory



Portion of disk can be used as extension of main memory to store inactive processes in case of shortage of memory - swap area

a.k.a. virtual memory

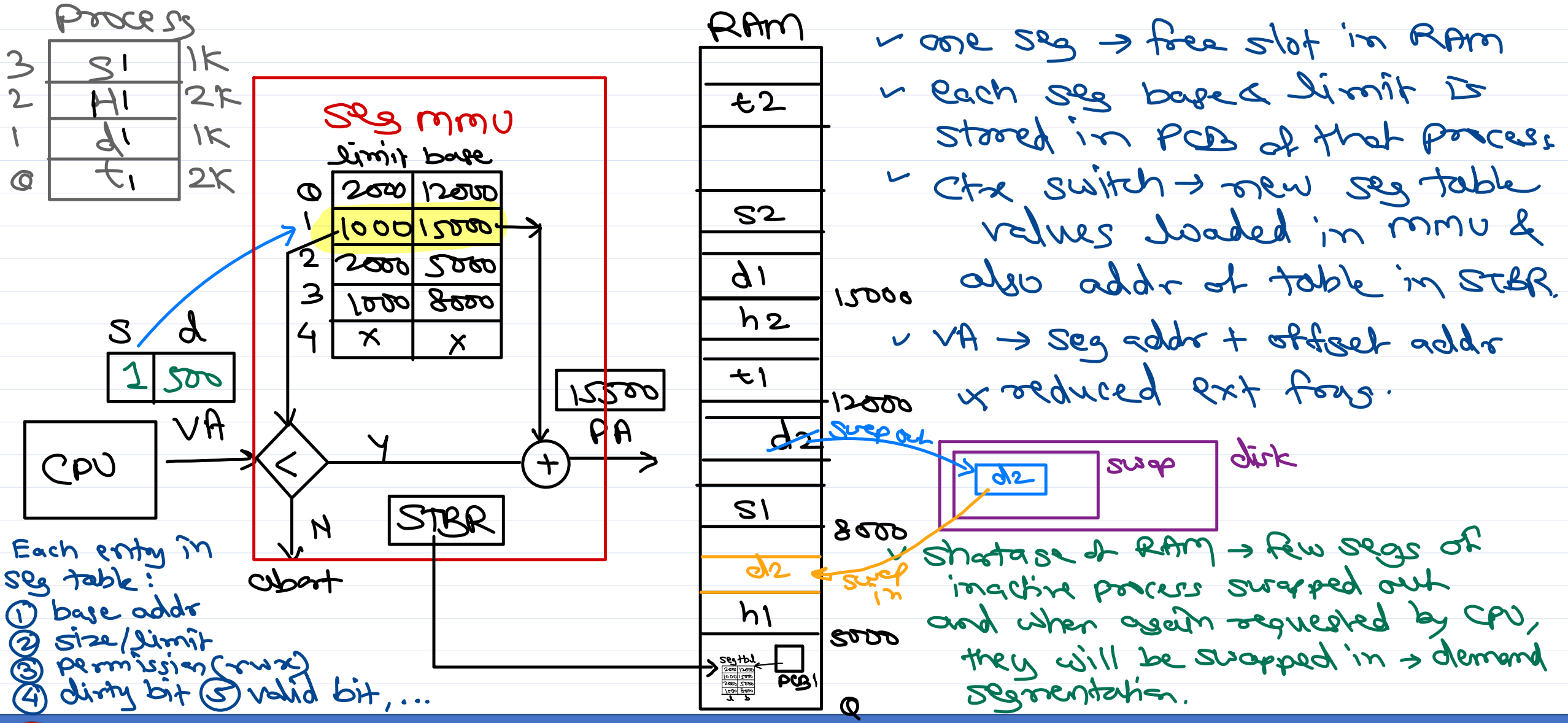
RAM ⇄ (swap out / swap in) Swap area

- Swap file (e.g. Windows) c:\pagefile.sys
- Swap part (e.g. Linux)

OS schedulers

1. job sched → long term sched
2. cpu sched → short term sched
3. swapper → medium term sched

Segmentation

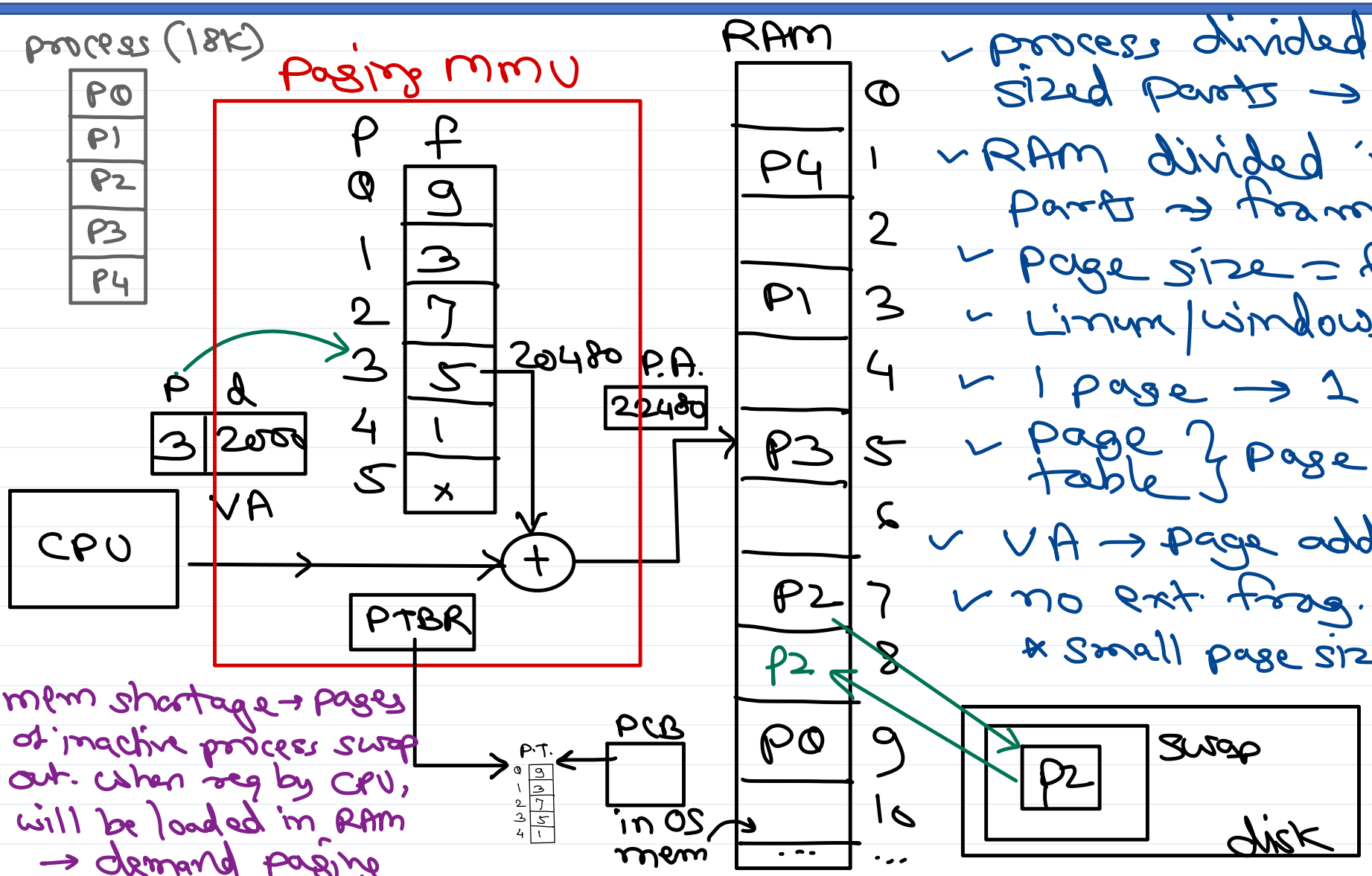


Each entry in seg table:
① base addr
② size/limit
③ permission (rwx)
④ dirty bit ⑤ valid bit, ...

- one seg → free slot in RAM
- each seg base & limit is stored in PCB of that process.
- Ctx switch → new seg table values loaded in mmu & also addr of table in STBR.
- VA → seg addr + offset addr
  - reduced ext frag.

Shortage of RAM → few segs of inactive process swapped out and when again requested by CPU, they will be swapped in → demand segmentation.

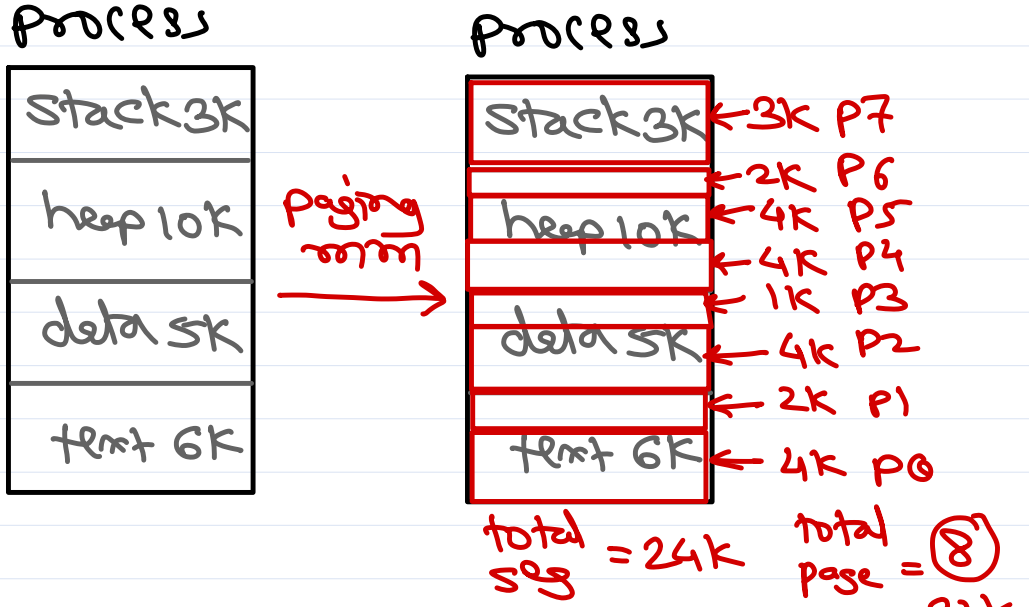
Paging



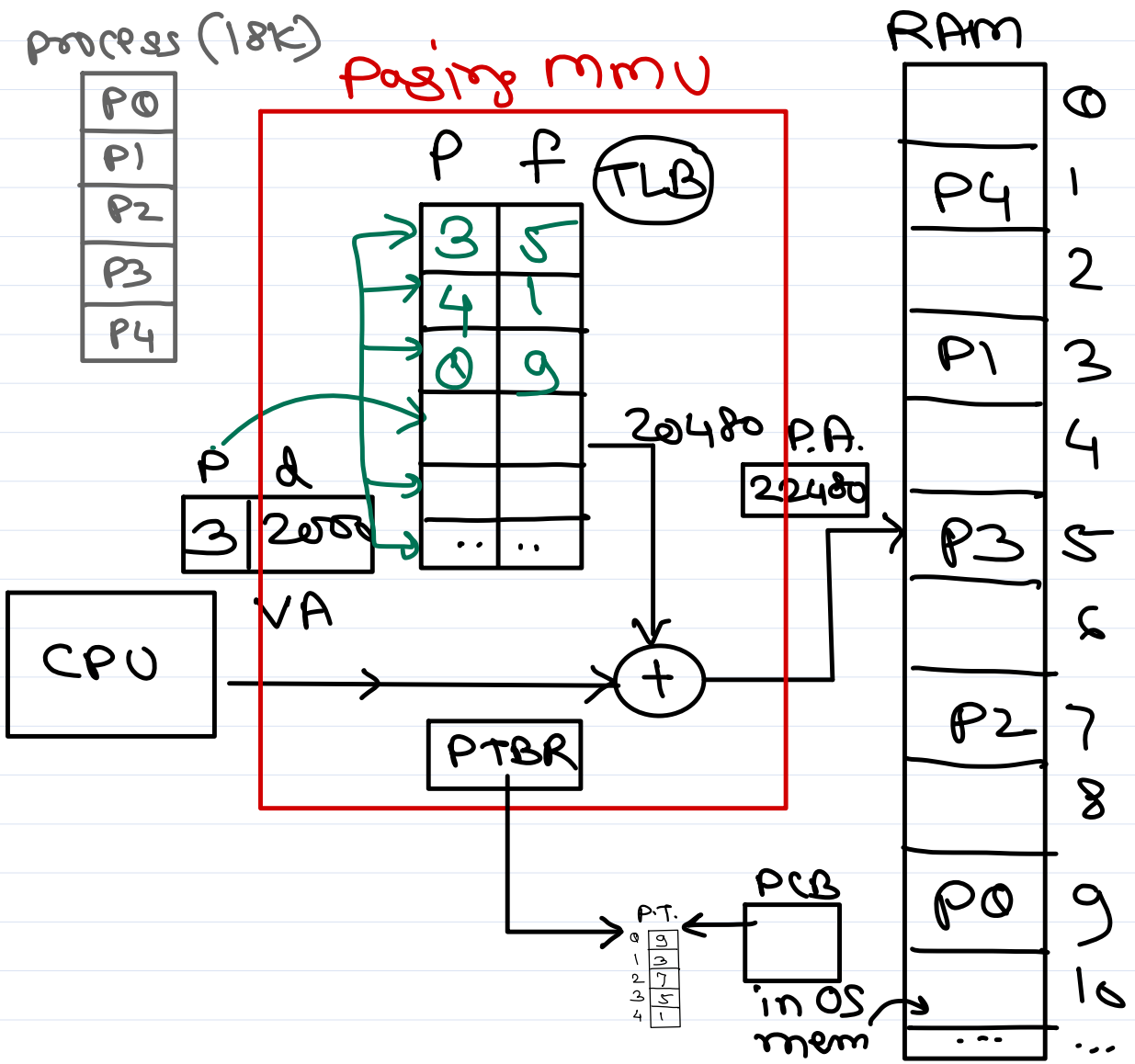
- process divided into small equal sized parts → pages.
- RAM divided into small equal parts → frames.
- page size = frame size.
- Linux/windows: page size = 4KB.
- 1 page → 1 empty frame.
- page table } page addr → frame addr
- VA → page addr + offset addr.
- no ext. frag. x internal frag.
  - * small page size ⇒ less int. frag but bigger page table.

Thrashing: more time spent in swapping (disk IO) than actual CPU execution. Overall system is slow.
Soln: Increase RAM.

Segmented Paging



TLB

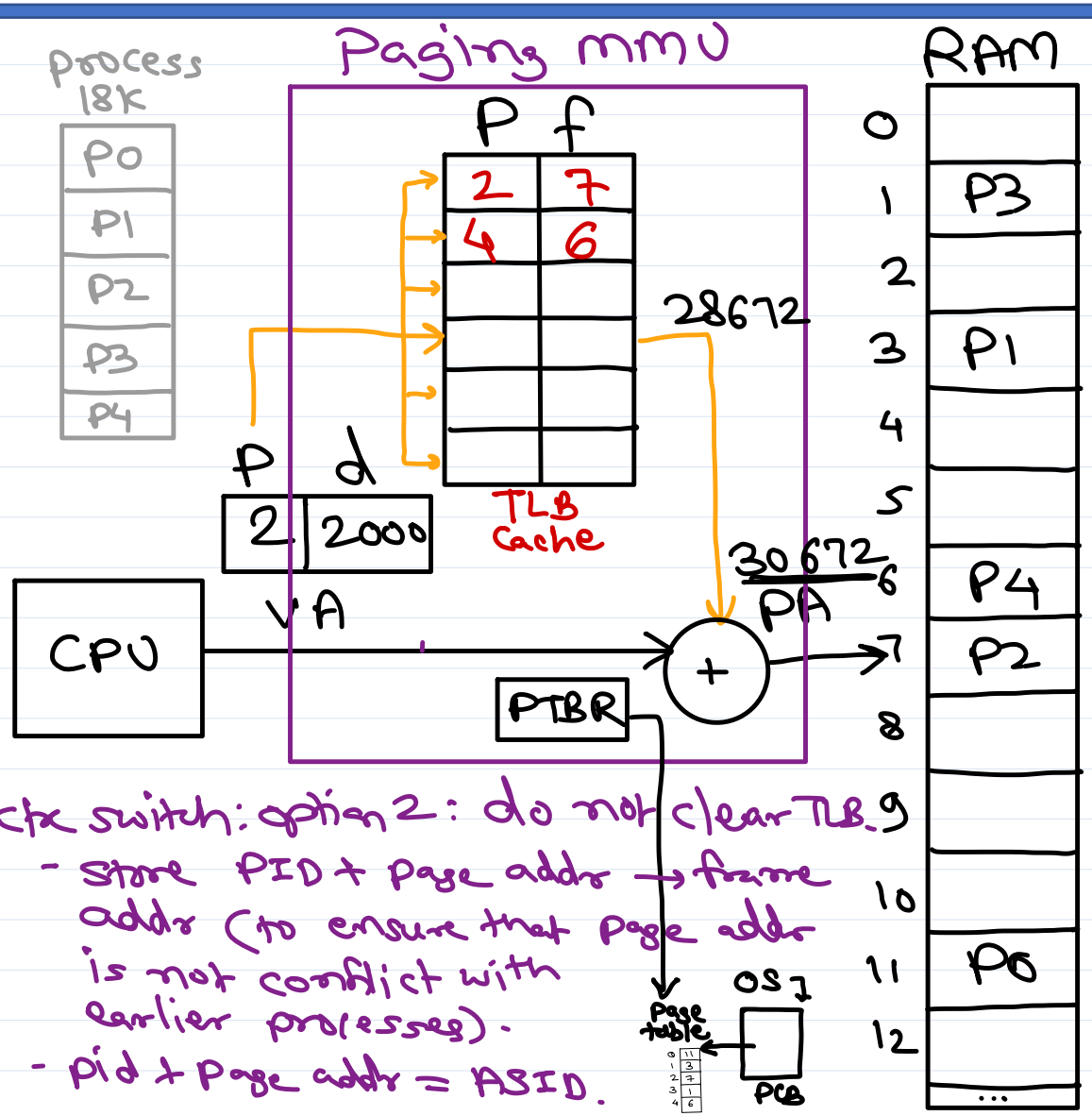


Translation Lookaside Buffer.
- high speed associative cache memory.

Page table entry :

1. frame addr
2. permission (r-w)
3. valid bit
4. dirty bit

TLB cache

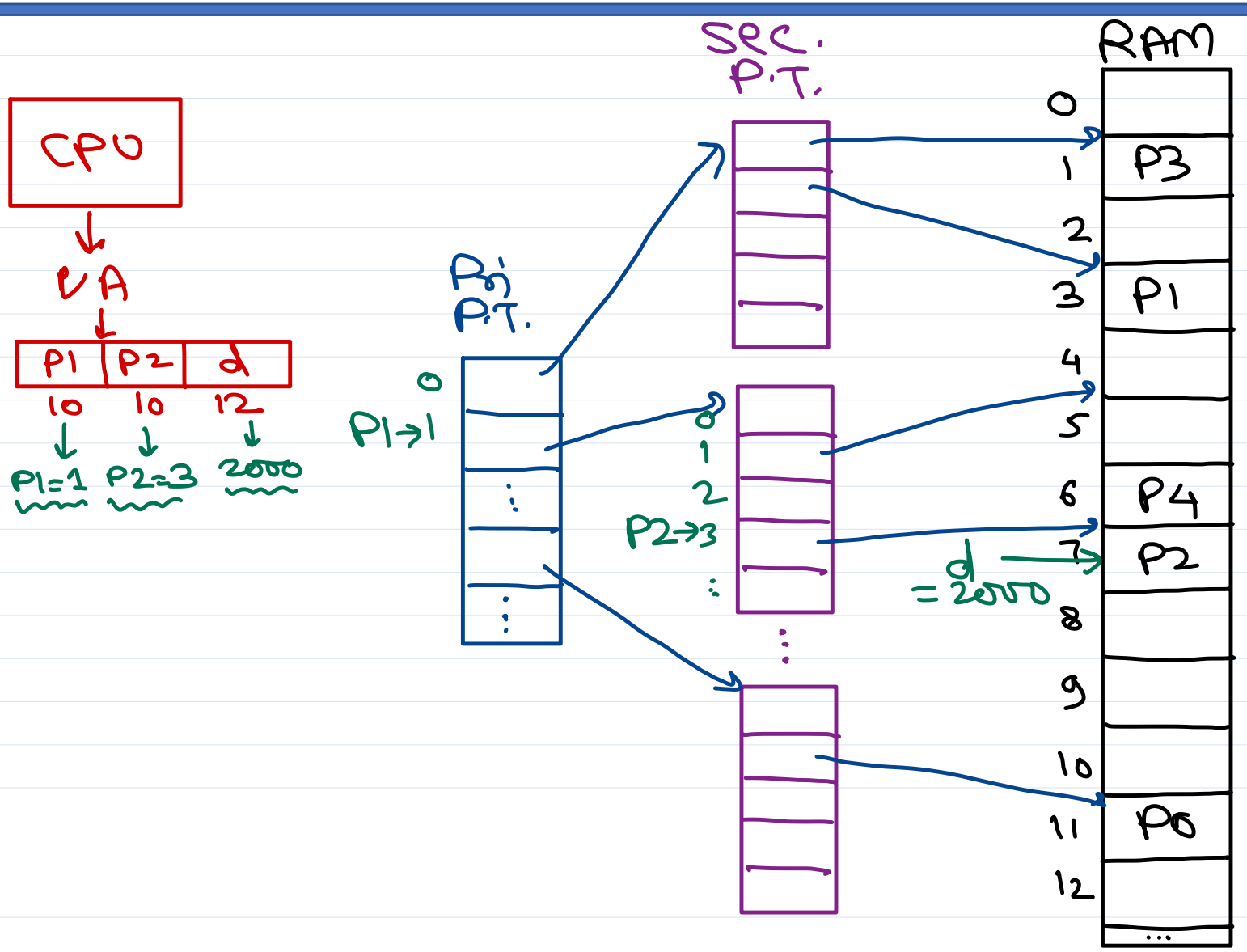


* Paging MMU = TLB Cache.
* TLB cache = High-Speed associative Cache.
  - ✓ Associative = page addr & frame addr mapping
  - ✓ High Speed = page addr compared with all entries at once to find corresponding frame addr
  - ✓ Cache = Limited entries (32/64 entries).
    - If page addr found (TLB hit), frame addr is retrieved.
    - If page addr not found (TLB miss), the process page table is read (using PTBR) and actual entry is copied into TLB cache.
    - If TLB is full, least recently used (LRU) entry is overwritten. Thus contains only recent entries.

Ctx switch: option 1: flush/clear TLB.
- New process CPU req will be TLB miss and then filled from process's page table (in RAM)

Ctx switch: option 2: do not clear TLB.
- store PID + page addr → frame addr (to ensure that page addr is not conflict with earlier processes).
- pid + page addr = ASID.

Two level paging



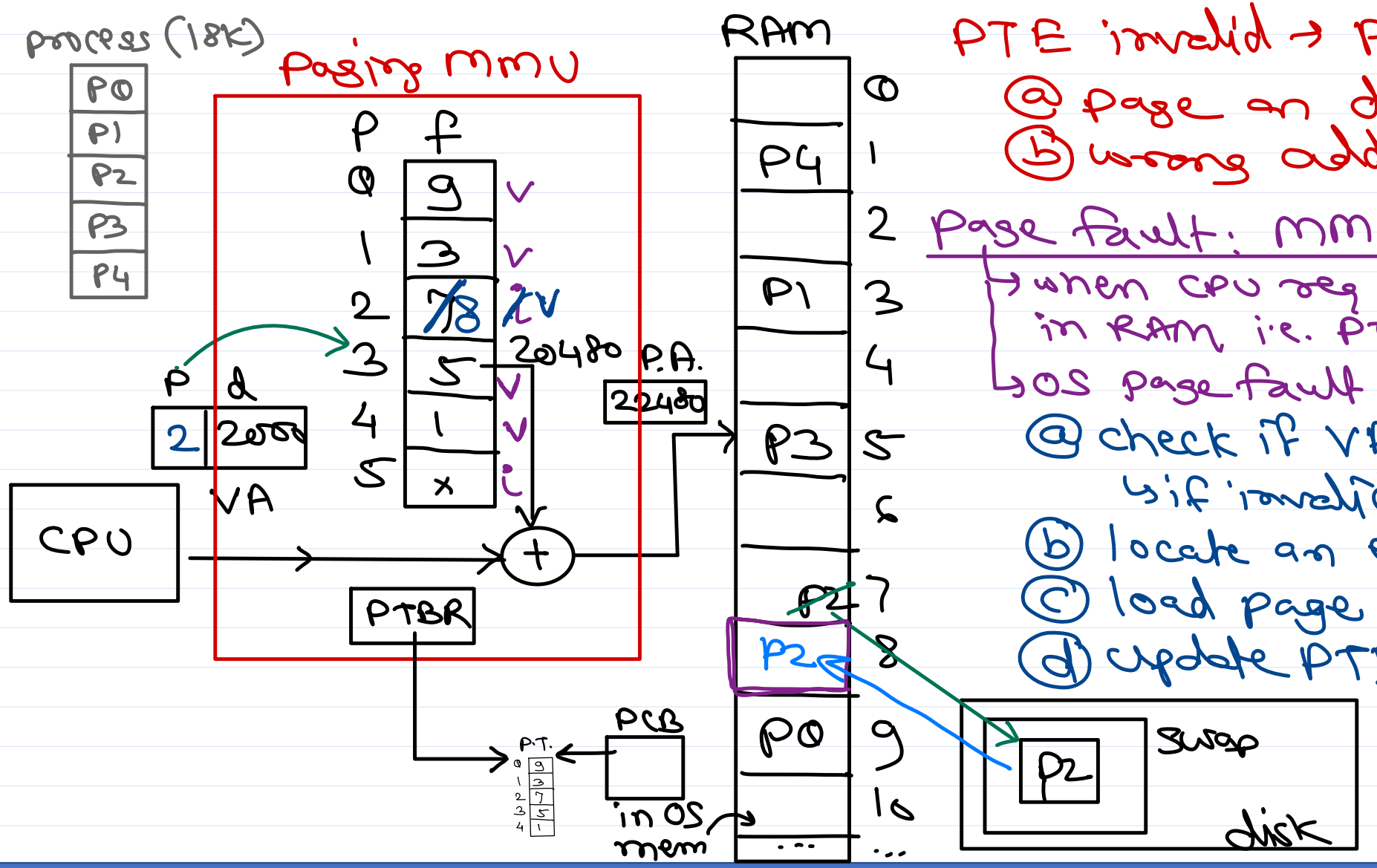
max virtual addresses
= 1024 × 1024 × 4 × 1024
= 4 GB

So virtual addr space of 32-bit processes = 4 GB

Pri PT a.k.a. → Page directory in x86 / L1 page table in ARM

Sec PT a.k.a. → Page table in x86 / L2 page table in ARM

Page Fault



PTE invalid → page not in RAM.

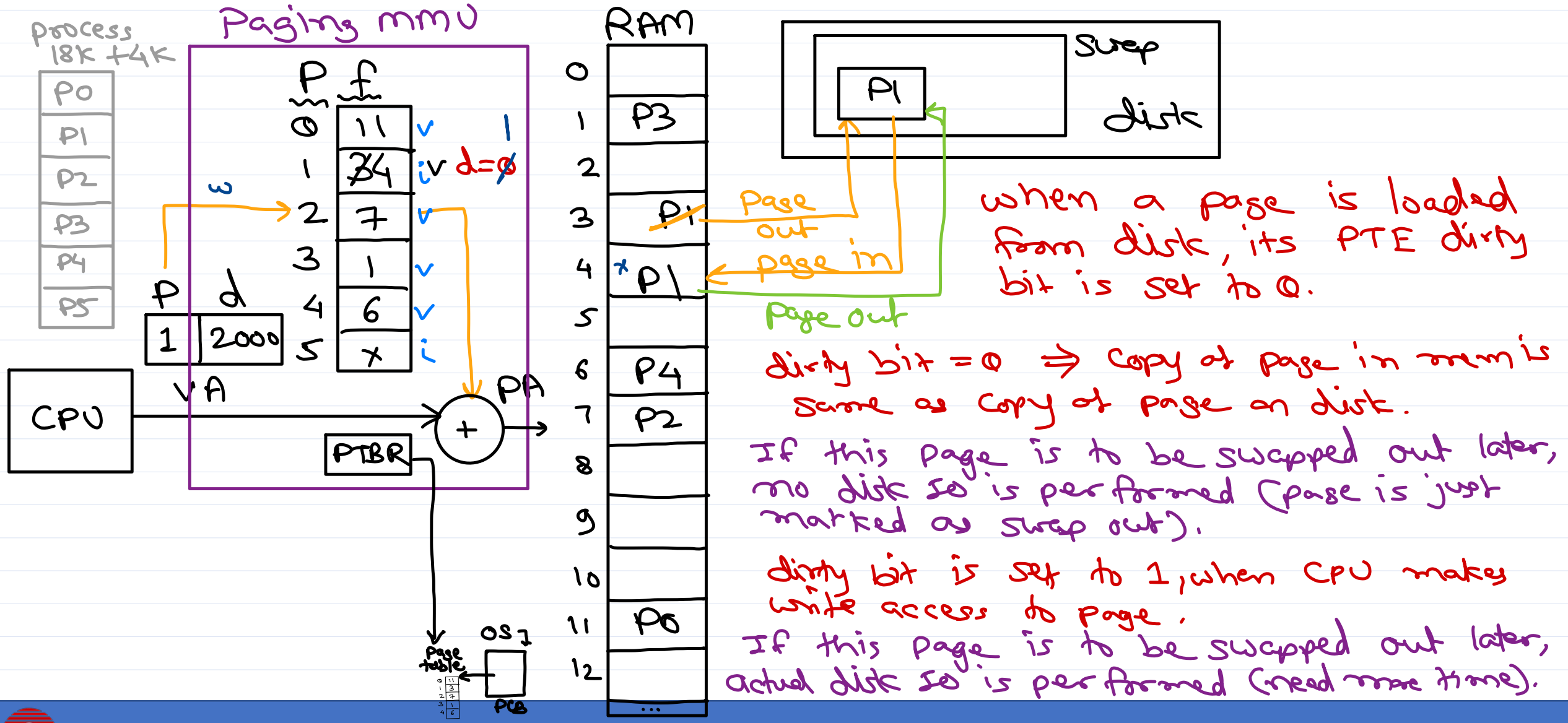
ⓐ page on disk/swap

ⓑ wrong addr (page not in proc)

Page fault: MMU interrupt/exception.

- when CPU req a page that is not in RAM, i.e. PTE is 'invalid'.
- OS page fault handler executed.
  - ⓐ check if VA is valid (dangling ptr).
    - if invalid, abort process.
  - ⓑ locate an empty frame.
  - ⓒ load page in that frame.
  - ⓓ update PTE.
  - ⓔ restart instruct at which fault occurred.

Dirty bit



when a page is loaded from disk, its PTE dirty bit is set to 0.

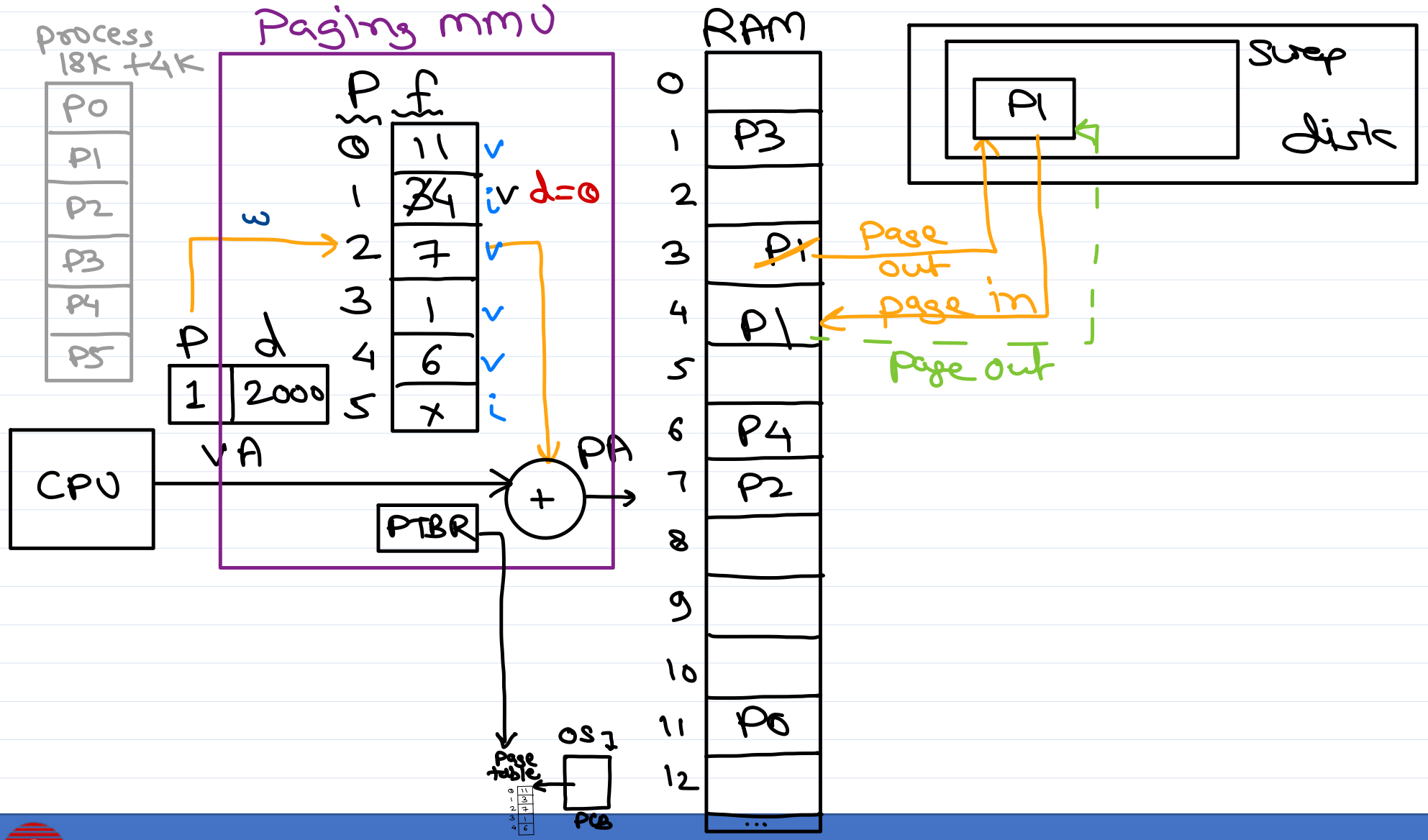
dirty bit = 0 ⇒ Copy of page in mem is same as copy of page on disk.

If this page is to be swapped out later, no disk IO is performed (page is just marked as swap out).

dirty bit is set to 1, when CPU makes write access to page.

If this page is to be swapped out later, actual disk IO is performed (need more time).

Dirty bit


process
18K + 4K
P0
P1
P2
P3
P4
P5
Paging mmu
P
f
0
1
2
3
4
5
11
34
7
1
6
x
v
iv
i
d=0
w
P
d
1
2000
VA
CPU
PA
+
PTBR
Page table
OS
PCB
RAM
0
1
2
3
4
5
6
7
8
9
10
11
12
P3
P1
P1
P4
P2
P0
Swap
disk
P1
Page out
page in
Page out

Thank you!

Nilesh Ghule <nilesh@sunbeaminfo.com>

Operating System Concepts

Agenda

- Deadlock
- Memory Management concepts
- Contiguous allocation
- Segmentation
- Paging
    - TLB cache
    - Two-level paging
    - Demand paging
    - Thrashing
    - Page fault handling
    - Page replacement algorithms
    - Dirty bit

Deadlock

- Refer yesterday slides

Memory Management

- In multi-programming OS, multiple programs are loaded in memory.
- RAM memory should be divided for multiple processes running concurrently.
- Memory Mgmt scheme used by any OS depends on the MMU hardware used in the machine.
- There are three memory management schemes are available (as per MMU hardware).
    1. Contiguous Allocation
    2. Segmentation
    3. Paging

Contiguous Allocation

Fixed Partition

- RAM is divided into fixed sized partitions.
- This method is easy to implement.
- Number of processes are limited to number of partitions.
- Size of process is limited to size of partition.
- If process is not utilizing entire partition allocated to it, the remaining memory is wasted. This is called as "internal fragmentation".

Dynamic Partition

- Memory is allocated to each process as per its availability in the RAM. After allocation and deallocation of few processes, RAM will have few used slots and few free slots.
- OS keep track of free slots in form of a table.
- For any new process, OS use one of the following mechanism to allocate the free slot.
    - First Fit: Allocate first free slot which can accommodate the process.
    - Best Fit: Allocate that free slot to the process in which minimum free space will remain.
    - Worst Fit: Allocate that free slot to the process in which maximum free space will remain.
- Statistically it is proven that First fit is faster algo; while best fit provides better memory utilization.
- Memory info (physical base address and size) of each process is stored in its PCB and will be loaded into MMU registers (base & limit) during context switch.
- CPU request virtual address (address of the process) and is converted into physical address by MMU as shown in diag.
- If invalid virtual address is requested by the CPU, process will be terminated.
- If amount of memory required for a process is available but not contiguous, then it is called as "external fragmentation".
- To resolve this problem, processes in memory can be shifted/moved so that max contiguous free space will be available. This is called as "compaction".

Virtual Memory

- The portion of the hard disk which is used by OS as an extension of RAM, is called as "virtual memory".
- If sufficient RAM is not available to execute a new program or grow existing process, then some of the inactive process is shifted from main memory (RAM), so that new program can execute in RAM (or existing process can grow). It is also called as "swap area" or "swap space".

- Shifting a process from RAM to swap area is called as "swap out" and shifting a process from swap to RAM is called as "swap in".
- In few OS, swap area is created in form of a partition. E.g. UNIX, Linux, ...
- In few OS, swap area is created in form of a file E.g. Windows (pagefile.sys), ...
- Virtual memory advantages:
    - Can execute more number of programs.
    - Can execute bigger sized programs.

Segmentation

- Instead of allocating contiguous memory for the whole process, contiguous memory for each segment can be allocated. This scheme is known as "segmentation".
- Since process does not need contiguous memory for entire process, external fragmentation will be reduced.
- In this scheme, PCB is associated with a segment table which contains base and limit (size) of each segment of the process.
- During context switch these values will be loaded into MMU segment table.
- CPU request virtual address in form of segment address and offset address.
- Based on segment address appropriate base-limit pair from MMU is used to calculate physical address as shown in diag.
- MMU also contains STBR register which contains address of current process's segment table in the RAM.

Demand Segmentation

- If virtual memory concept is used along with segmentation scheme, in case low memory, OS may swap out a segment of inactive process.
- When that process again start executing and ask for same segment (swapped out), the segment will be loaded back in the RAM. This is called as "demand segmentation".
- Each entry of the segment table contains base & limit of a segment. It also contains additional bits like segment permissions, valid bit, dirty bit, etc
- If segment is present in main memory, its entry in seg table is said to be valid (v=1). If segment is swapped out, its entry in segment table is said to be invalid (v=0).

Paging

- RAM is divided into small equal sized partitions called as "frames" / "physical pages".
- Process is divided into small equal sized parts called as "pages" or "logical/virtual pages".
- page size = frame size.
- One page is allocated to one empty frame.

- OS keep track of free frames in form of a linked list.
- Each PCB is associated with a table storing mapping of page address to frame address. This table is called as "page table".
- During context switch this table is loaded into MMU.
- CPU requests a virtual address in form of page address and offset address. It will be converted into physical address as shown in diag.
- MMU also contains a PTBR, which keeps address of page table in RAM.
- If a page is not utilizing entire frame allocated to it (i.e. page contents are less than frame size), then it is called as "internal fragmentation".
- Frame size can be configured in the hardware. It can be 1KB, 2KB or 4KB, ...
- Typical Linux and Windows OS use page size = 4KB.

Page table entry

- Each PTE is of 32-bit (on x86 arch) and it contains
  - Frame address
  - Permissions (read or write)
  - Validity bit
  - Dirty bit
  - ...

TLB (Translation Look-Aside Buffer) Cache

- TLB is high-speed associative cache memory used for address translation in paging MMU.
- TLB has limited entries (e.g. in P6 arch TLB has 32 entries) storing recently translated page address and frame address mappings.
- The page address given by CPU, will be compared at once with all the entries in TLB and corresponding frame address is found.
- If frame address is found (TLB hit), then it is used to calculate actual physical address in RAM (as shown in diag).
- If frame address is not found (TLB miss), then PTBR is used to access actual page table of the process in the RAM (associated with PCB). Then page-frame address mapping is copied into TLB and thus physical address is calculated.
- If CPU requests for the same page again, its address will be found in the TLB and translation will be faster

Two Level Paging

- Primary page table has number of entries and each entry point to the secondary page table page.
- Secondary page table has number of entries and each entry point to the frame allocated for the process.

- Virtual address requested by a process is 32 bits including
  - p1 (10 bits) -> Primary page table index/addr
  - p2 (10 bits) -> Secondary page table index/addr
  - d (12 bits) -> Frame offset

Demand Paging

- When virtual memory is used with paging memory management, pages can be swapped out in case of low memory.
- The pages will be loaded into main memory, when they are requested by the CPU. This is called as "demand paging".
  - Swapped out pages
  - Pages from program image (executable file on disk)
  - Dynamically allocated pages

Virtual pages vs Logical pages

- By default all pages of user space process can be swapped out/in. This may change physical address of the page. All such pages whose physical address may change are referred as "Virtual pages".
- Few kernel pages are never swapped out. So their physical address remains same forever. All such pages whose physical address will not change are referred as "Logical pages".

Thrashing

- If number of programs are running in comparatively smaller RAM, a lot of system time will be spent into page swapping (paging) activity.
- Due to this overall system performance is reduced.
- The problem can be solved by increasing RAM size in the machine.

Page Fault

- Each page table entry contains frame address, permissions, dirty bit, valid bit, etc.
- If page is present in main memory its page table entry is valid (valid bit = 1).
- If page is not present in main memory, its page table entry is not valid (valid bit = 0).
- This is possible due to one of the following reasons:
  - Page address is not valid (dangling pointer).

    - Page is on disk/swapped out.
    - Page is not yet allocated.
- If CPU requests a page that is not present in main memory (i.e. page table entry valid bit=0), then "page fault" occurs.
- Then OS's page fault exception handler is invoked, which handles page faults as follows:
    1. Check virtual address due to which page fault occured. If it is not valid (i.e. dangling pointer), terminate the process (sending SEGV signal). (Validity fault).
    2. Check if read-write operation is permitted on the address. If not, terminate the process (sending SEGV signal). (Protection fault).
    3. If virtual address is valid (i.e. page is swapped out), then locate one empty frame in the RAM.
    4. If page is on swap device or hard disk, swap in the page in that frame.
    5. Update page table entry i.e. add new frame address and valid bit = 1 into PTE.
    6. Restart the instruction for which page fault occurred.

Page Replacement Algorithms

- While handling page fault if no empty frame found (step 3), then some page of any process need to be swapped out. This page is called as "victim" page.
- The algorithm used to decide the victim page is called as "page replacement algorithm".
- There are three important page replacement algorithms.
    - FIFO
    - Optimal
    - LRU

FIFO

- The page brought in memory first, will be swapped out first.
- Sometimes in this algorithm, if number of frames are increased, number of page faults also increase.
- This abnormal behaviour is called as "Belady's Anomaly".

OPTIMAL

- The page not required in near future is swapped out.
- This algorithm gives minimum number of page faults.
- This algorithm is not practically implementable.

LRU

- The page which not used for longer duration will be swapped out.
- This algorithm is used in most OS like Linux, Windows, ...
- LRU mechanism is implemented using "stack based approach" or "counter based approach".
- This makes algorithm implementation slower.
- Approximate LRU algorithm close to LRU, however is much faster.

Dirty Bit

- Each entry in page table has a dirty bit.
- When page is swapped in, dirty bit is set to 0.
- When write operation is performed on any page, its dirty bit is set to 1. It indicate that copy of the page in RAM differ from the copy in swap area.
- When such page need to be swapped out again, OS check its dirty bit. If bit=0 (page is not modified) actual disk IO is skipped and improves performance of paging operation.
- If bit=1 (page is modified), page is physically overwritten on its older copy in the swap area.